SWIFTSHIP TRACKER

PROJECT DOCUMENTATION

Team ID	SWTID-2026-4838
Team Size	5
Team Leader	Shalini P
Team Members	Subalaksmi c Vaishnavi R Selvameena E Atchaya K
College Name	Government College of Engineering, Thanjavur.
Academic Year	2026-2027

Table of Contents

1. Project Overview 3
1.1 Introduction 3
1.2 Purpose of the Project 3
2. Objectives 4
2.1 Project Scope 4
2.2 Project Objectives 4
2.3 Business Value 4
3. Phase-wise Implementation 6
3.1 Implementation Overview 6
3.2 Phase 1: Requirement Analysis & Planning 6
3.3 Phase 2: Salesforce Development: Backend & Configurations 16
3.4 Phase 3: UI/UX Development & Customization 33
3.5 Phase 4: Data Migration, Testing & Security 42
4. Phase 5: Deployment, Documentation & Maintenance 47
4.1 Deployment Strategy 47
4.2 Maintenance, Monitoring & Troubleshooting 47
4.3 Documentation Summary 47
5. Conclusion 48
5.1 Project Learnings and Outcomes 48
5.2 Advantages 48
5.3 Limitations 48
5.4 Future Scope 48
Appendix 50

1. Project Overview

1.1 Introduction

The SwiftShip Tracker is a Salesforce CRM solution developed to simplify parcel booking, shipment tracking, and delivery management. It supports the complete parcel lifecycle, from booking and dispatch through tracking and delivery confirmation. Customers can access shipment information, while delivery agents can update parcel details and delivery status.

The system provides a centralized Salesforce-based parcel management solution that enables customers to book parcels, track delivery status, and manage shipments through an intuitive interface and conversational AI support. It manages Parcel, Delivery, Sender, and Receiver information using Salesforce custom objects and relationships. Flows automate parcel operations and notifications, while Agentforce AI and Prompt Builder enable customers to retrieve parcel information and interact with the system conversationally. Experience Cloud, security controls, reports, and dashboards provide secure customer access and operational visibility.

1.2 Purpose of the Project

The purpose of SwiftShip Tracker is to:

- Centralize parcel and delivery information.
- Simplify parcel booking and tracking.
- Provide real-time shipment visibility.
- Automate delivery updates and customer notifications.
- Enable conversational parcel tracking using Agentforce AI and Prompt Builder.
- Improve agent efficiency through Salesforce Flows and dashboards.
- Provide secure, role-based access to customers, agents, support staff, and administrators.

2. Objectives

2.1 Project Scope

1. Build a scalable CRM solution for parcel delivery businesses to manage customer bookings, shipments, and delivery operations efficiently.
2. Support the end-to-end parcel lifecycle: parcel booking, dispatch, tracking, and delivery confirmation.
3. Include data model design, automation (Flows, Apex, Email Alerts), an AI-powered Prompt Builder, and reporting dashboards for effective management.
4. Provide role-based access for admins, delivery agents, and customers through Salesforce profiles, permission sets, and Experience Cloud.
5. Ensure real-time parcel visibility for customers and simplified operational tracking for agents through AI-integrated conversational support.

2.2 Project Objectives

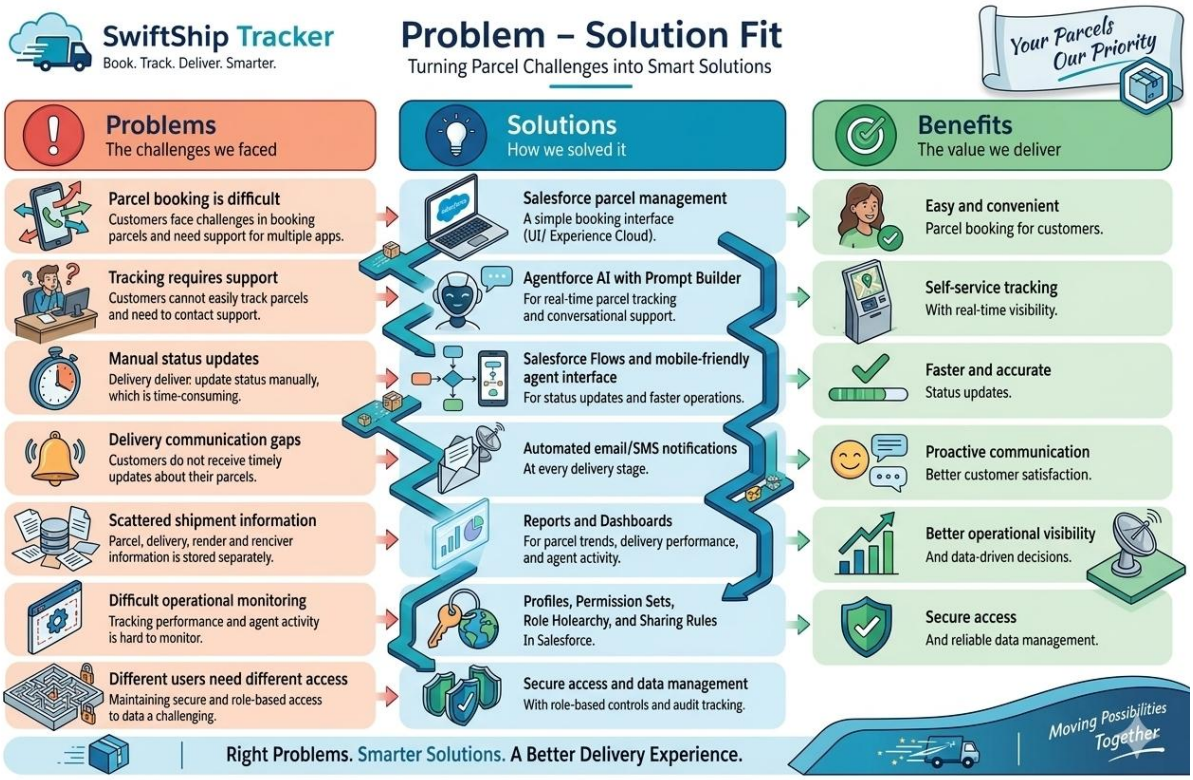
- **Streamline operations:** automate booking, tracking, and delivery notifications to reduce manual updates.
- **Improve customer satisfaction:** provide transparent, real-time shipment status and self-service tracking options.
- **Enhance agent efficiency:** offer centralized dashboards and guided Flows for quick status updates.
- **Enable business insights:** provide reports and dashboards showing parcel trends, delivery performance, and agent activity.
- **Ensure security and reliability:** implement Salesforce role hierarchies, sharing rules, and audit-based data access.

2.3 Business Value

The table below summarizes the business problems addressed by SwiftShip Tracker and the benefit delivered by each solution component.

Problem	Solution	Benefit
Parcel booking is difficult	Salesforce parcel management	Centralized booking
Tracking requires support	Agentforce AI + Prompt Builder	Self-service tracking
Manual status updates	Salesforce Flows	Automated processes
Delivery communication gaps	Email Alerts	Timely notifications
Scattered shipment information	Parcel, Delivery, Sender and Receiver objects	Structured data
Difficult operational monitoring	Reports and Dashboards	Better visibility
Different users need different access	Profiles, Roles and Permission Sets	Secure access

Figure 1: Problem–Solution Fit



3. Phase-wise Implementation

3.1 Implementation Overview

The project was implemented in five phases, following the sequence Requirement, Development, UI, Testing, and Security. Phases 1 to 4 are detailed in this section, and Phase 5 is covered in Section 4.

Phase	Title	Focus
Phase 1	Requirement Analysis & Planning	Problem definition, requirements, solution design, planning, data model and security model design
Phase 2	Salesforce Development: Backend & Configurations	Developer account, custom objects, tabs, fields and relationships, Prompt Builder template, Flow
Phase 3	UI/UX Development & Customization	Permission sets, agent user permissions, Agentforce org setup, agent, subagent and action configuration
Phase 4	Data Migration, Testing & Security	Verification of agent, topic, flow and data; test results; security validation
Phase 5	Deployment, Documentation & Maintenance	Deployment scope, maintenance, monitoring and troubleshooting, documentation

3.2 Phase 1: Requirement Analysis & Planning

Objective: Customers often face delays and confusion while booking, tracking, and managing parcel deliveries because they need to call support or use multiple apps. A conversational AI agent can simplify this by allowing customers to book parcels, get live tracking, and manage deliveries in one place.

The idea behind SwiftShip Tracker is to provide a structured Salesforce-based solution for handling parcel-related information and inquiries. The system uses Salesforce objects to maintain parcel and related information, while Agentforce is used to provide a conversational interface for accessing the required parcel details.

The project combines Salesforce data management with automation and AI-based interaction. This allows the system to organize parcel information and provide users with a convenient way to make parcel-related inquiries.

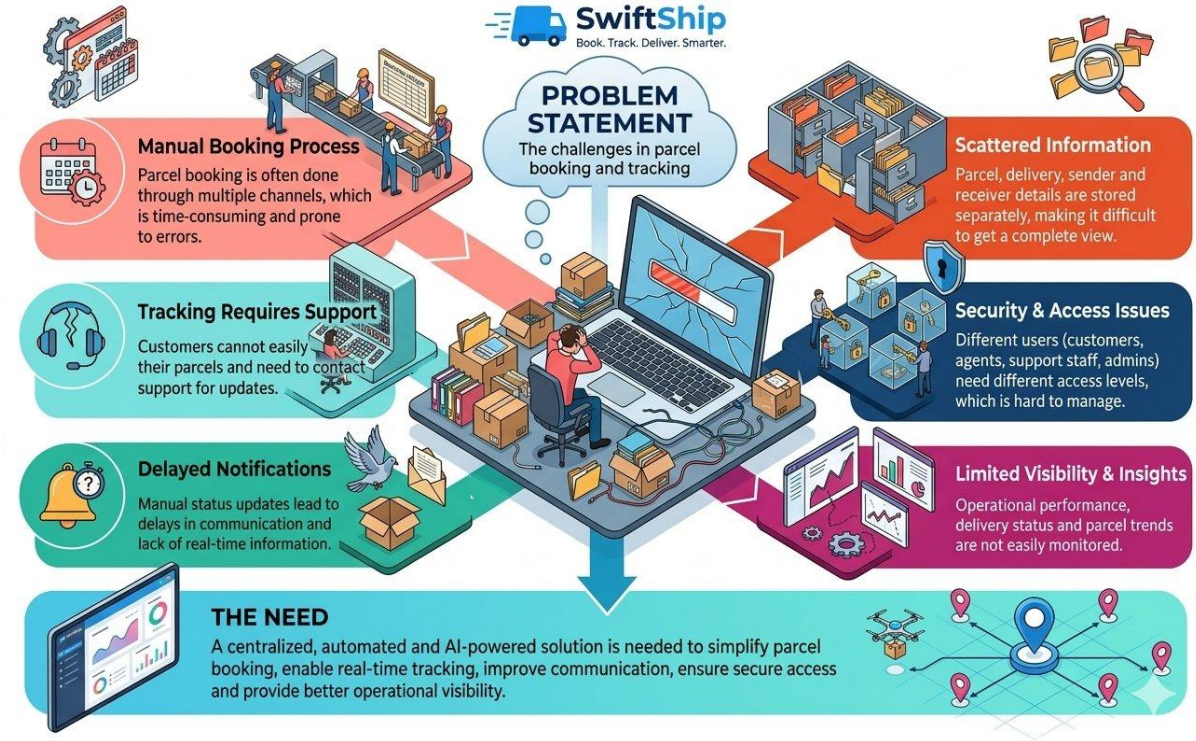
3.2.1 Ideation

Problem Statements

PS	I am (Customer)	I'm trying to	But	Because	Which makes me feel
PS-1	Customer / Sender	book a parcel easily	booking requires support assistance or multiple applications	parcel operations are not centralized	frustrated
PS-2	Customer / Sender	track my parcel	I do not have a simple real-time tracking experience	shipment information is difficult to access	confused

PS	I am (Customer)	I'm trying to	But	Because	Which makes me feel
PS-3	Delivery Agent	update parcel status	status updates require manual handling	there is a need for centralized operational tracking	concerned
PS-4	Customer	receive delivery updates	notifications may not be available at every delivery stage	communication needs to be automated	uncertain
PS-5	Admin	monitor parcel operations	parcel flow and delivery performance are difficult to monitor	operational data needs centralized reporting	concerned

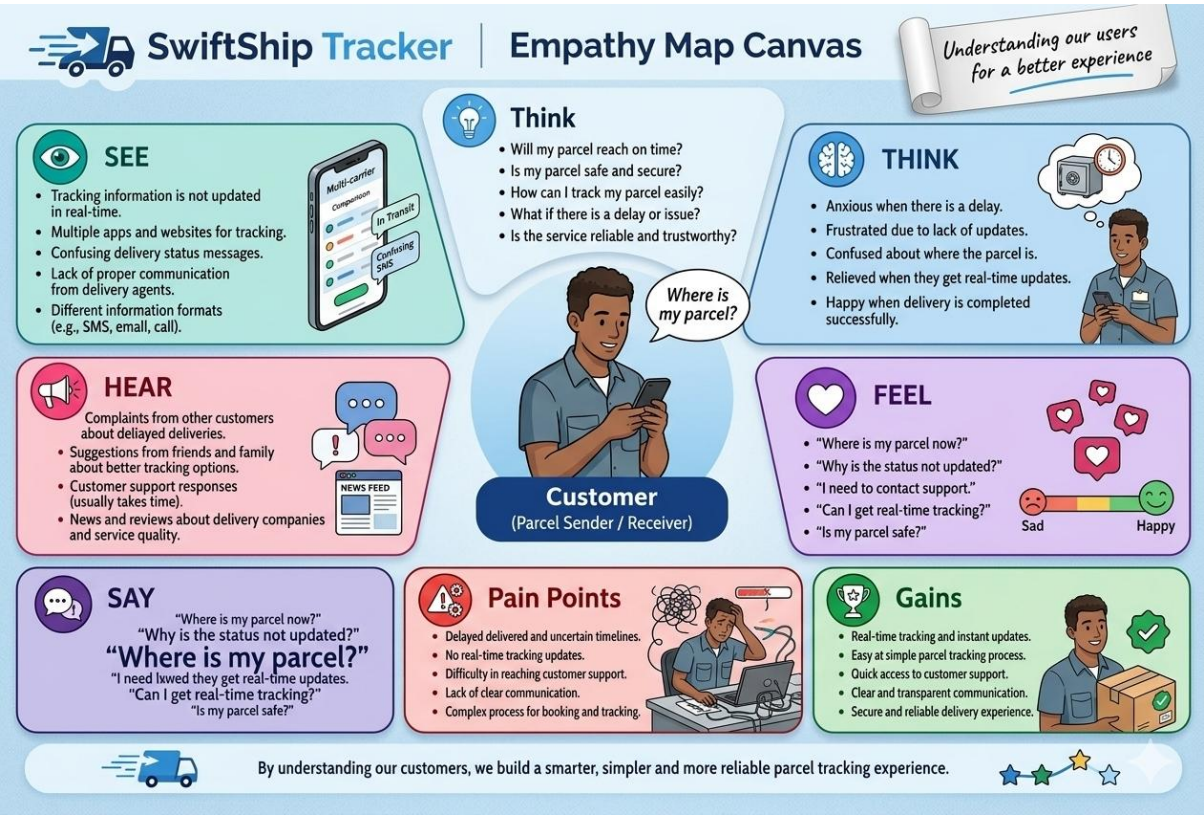
Figure 2: Problem Statements



Empathy Map Canvas (User Persona: Customer / Sender)

- **Think & Feel:** Wants a simple way to book and track parcels without repeatedly contacting support.
- **Hear:** Receives delivery information and status updates from the parcel service.
- **See:** Parcel details, tracking status, estimated delivery date, and delivery information.
- **Say & Do:** Books parcels, checks tracking information, and asks for delivery updates.
- **Pain:** Delays, confusion, lack of easy tracking, and dependence on support or multiple apps.
- **Gain:** Real-time visibility, self-service tracking, convenient booking, and timely notifications.

Figure 3: Empathy Map Canvas



Brainstorming and Idea Prioritization

Four ideas were considered: manual registers, spreadsheet tracking, a standalone parcel portal, and Salesforce CRM with Agentforce AI. Salesforce CRM was selected for the following reasons:

- Centralized parcel management
- Automation through Flows
- Real-time tracking visibility
- Conversational AI assistance
- Secure role-based access
- Reports and dashboards

Figure 4: Brainstorming and Idea Prioritization



3.2.2 Requirement Analysis

The requirement analysis identifies the information that needs to be maintained for the parcel tracking process. The project uses custom objects such as Parcel__c, Delivery__c, Sender__c, and Receiver__c to organize the required data.

The Parcel object contains important information such as Parcel ID, Status, Weight, Estimated Delivery Date, and Sender. The Delivery object maintains delivery-related information such as Current Location, Estimated Delivery, Parcel, and Sender. Sender and Receiver objects are used to maintain the corresponding contact and address information.

The requirements identified in this phase form the basis for creating the Salesforce data model and configuring the relationships between the objects.

Customer Journey Map

The customer journey covers the following stages: Parcel Booking, Parcel Record Creation, Dispatch & Shipment, In-Transit Tracking, Out for Delivery, Delivery Confirmation, Customer Notification, and Feedback / Support.

Figure 5: Customer Journey Map



Users Involved

- **Customers / Senders:** book parcels, track deliveries, and receive real-time updates through the portal or AI agent.
- **Delivery Agents:** manage assigned parcels, update delivery status, and ensure timely handover.
- **Customer Support Team:** handle delivery-related issues and respond to customer queries.
- **Admin Team:** configure the Salesforce environment, manage user roles and security settings, and monitor operational performance.
- **System Integrator / Developer:** implement automation, AI agent setup, and backend integrations.

Functional Requirements

FR No	Requirement
FR-1	Parcel booking and record creation
FR-2	Parcel status tracking
FR-3	Weight management
FR-4	Estimated delivery date management
FR-5	Sender information management
FR-6	Receiver information management
FR-7	Delivery and current-location tracking

FR No	Requirement
FR-8	Agent parcel status updates
FR-9	Automated customer notifications
FR-10	AI-powered parcel tracking
FR-11	Reports and dashboards
FR-12	Customer access through Experience Cloud

Key Functional Needs

- Seamless parcel booking and tracking interface for customers through a Salesforce Experience Site or agent assistance.
- Ability for agents to update parcel status (Booked, In Transit, Out for Delivery, Delivered) in real time.
- Automated notifications and email alerts to customers at every delivery stage.
- Sender and Receiver details management with address, contact, and email fields.
- AI agent integration (via Prompt Builder) for real-time parcel status queries and automated assistance.
- Reports and dashboards to monitor parcel flow, delivery performance, and agent activity.
- Role-based access, profiles, and permission sets for different user levels.

Non-Functional Requirements

NFR	Description
Usability	Intuitive Salesforce and customer-facing interface
Security	Role-based and field-level access control
Reliability	Accurate parcel status and delivery information
Performance	Timely Flow and Agentforce responses
Availability	Salesforce cloud-based access
Scalability	Supports growing parcel and delivery operations
Data Integrity	Controlled access and relationship-based parcel data
Maintainability	Flows, prompts, topics, and configurations can be updated

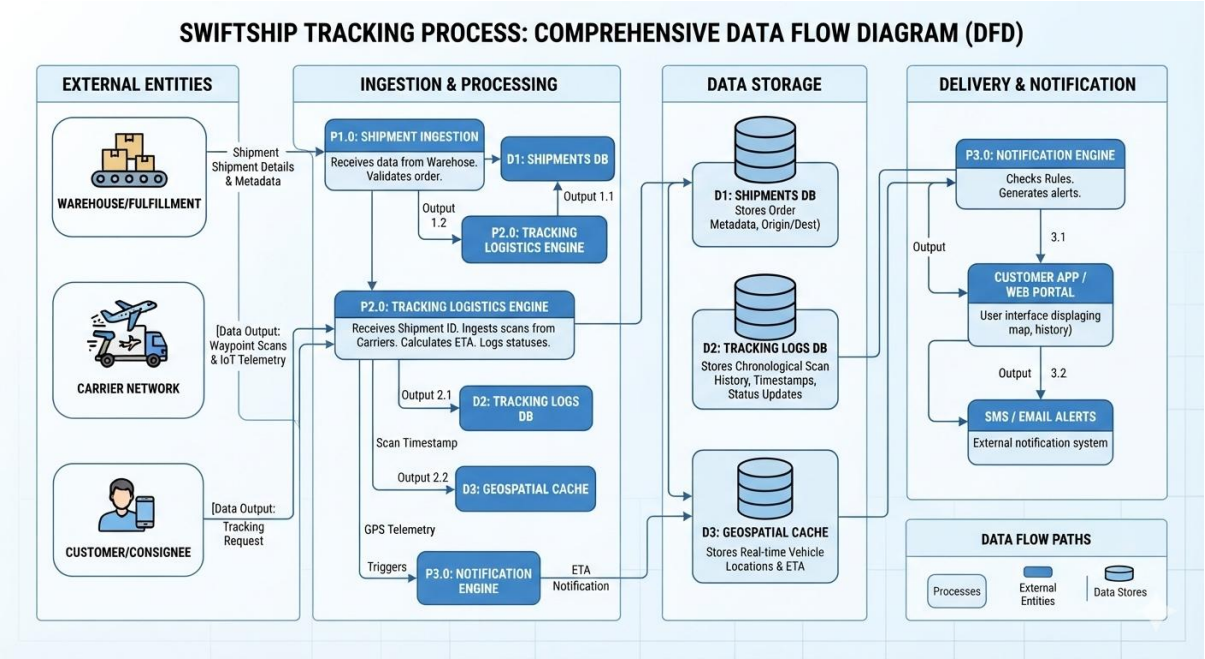
Data Flow Diagram

Parcel flow: Customer / Sender → Parcel Booking → Parcel Record → Sender + Receiver Information → Delivery Record → Parcel Status Update → Current Location & Estimated Delivery → Automated Notifications → Customer Tracking / Agentforce AI → Reports & Dashboards.

AI tracking flow: Customer Query → Agentforce AI → Prompt Builder (Retrieve Parcel Details) → Parcel Record → Status / Weight / Estimated Delivery → Conversational

Response. The Prompt Builder implementation retrieves Parcel ID, Status, Weight, and Estimated Delivery Date information for tracking responses.

Figure 6: Data Flow Diagram



Technology Stack

Layer	Technology
Platform	Salesforce Developer Edition
UI	Lightning App Page, Lightning Record Pages, Experience Cloud
Database	Salesforce Custom & Standard Objects
Business Logic	Salesforce Flows, Apex Classes
Automation	Record-Triggered Flows, Auto-Launched Flows, Scheduled Flows
AI	Agentforce AI, Prompt Builder
Notifications	Email Alerts & Email Templates
Processing	Batch Apex
Security	Profiles, Permission Sets, Role Hierarchy, Field-Level Security, Sharing Rules
Reporting	Reports & Dashboards

Salesforce Features and Tools Identified

- **Custom Objects:** Parcel__c, Delivery__c, Sender__c, Receiver__c.
- **Standard Objects:** Account, Contact, Case, Task, EmailMessage, User.
- **Automations:** Record-Triggered Flows, Auto-Launched Flows, Approval Processes, Scheduled Flows.

- **Apex:** Apex Classes for automated parcel operations and Batch Apex for periodic updates or notifications.
- **UI:** Lightning App Page (SwiftShip Tracker App), Dynamic Forms for Parcel Details, Lightning Record Pages, and Experience Cloud for customer access.
- **Agentforce AI & Prompt Builder:** an AI agent that fetches parcel data based on Parcel ID and responds to tracking or delivery status queries.
- **Email Services:** Custom Email Alerts and Templates for booking confirmations, delivery updates, and completion acknowledgments.
- **Security:** Profiles, Permission Sets, Role Hierarchies, Field-Level Security, and Sharing Rules.
- **Tools used:** Salesforce Developer Org, Prompt Builder, Salesforce Flow Builder, and Google Sheets / Docs for data planning and documentation.

3.2.3 Project Design

The system design defines how the different Salesforce components will work together. The custom objects form the data layer, while Salesforce Flow provides the required automation. Prompt Builder is used to process the retrieved parcel information, and Agentforce provides the conversational interaction layer.

The overall project structure can be represented as:

Salesforce Data → Flow Automation → Prompt Builder → Agentforce → User Response

This design provides a clear connection between the stored parcel information and the user-facing agent interaction.

Proposed Solution

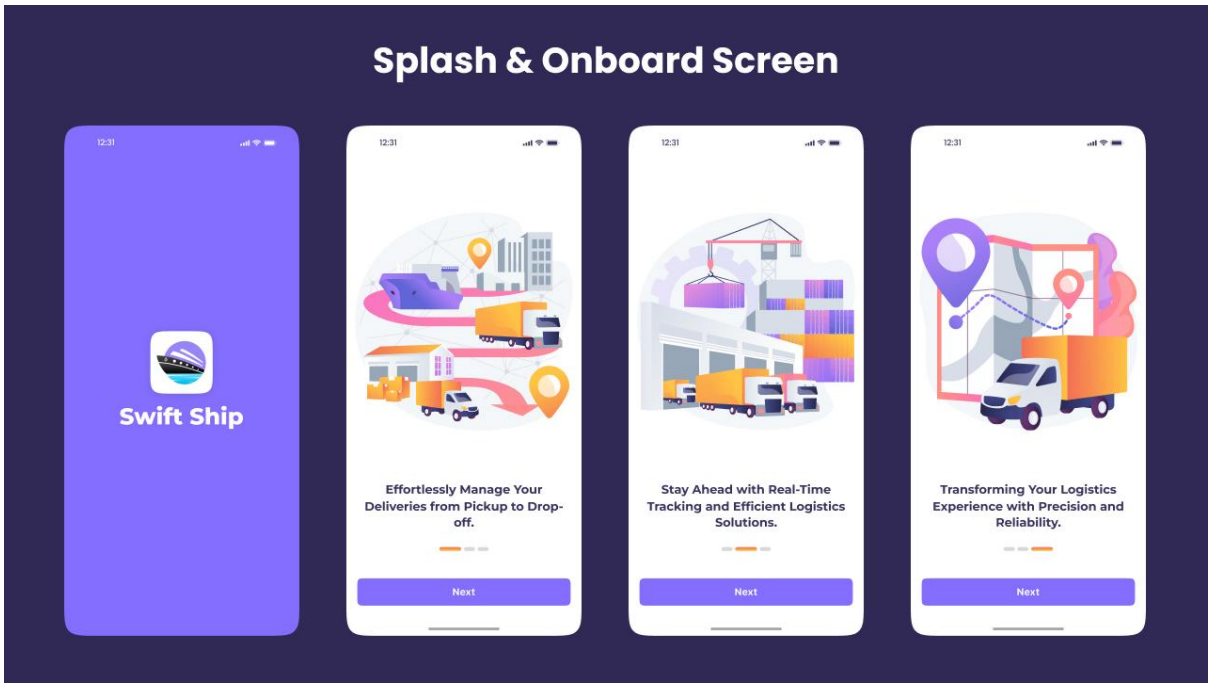
Parameter	Description
Problem	Difficult and inefficient parcel booking and tracking
Solution	Salesforce-based Parcel Management System
Innovation	Agentforce AI + Prompt Builder for conversational self-service tracking and automated Salesforce Flows
Impact	Centralized booking, real-time tracking visibility, timely notifications, and efficient delivery management
Business Model	Parcel, customer, delivery-agent, support, and administrator operations
Scalability	Supports growing parcel operations, Experience Cloud, and integration with external logistics systems

Solution Architecture

Users (Customer, Delivery Agent, Support Staff, Admin) access the solution through Salesforce Lightning and Experience Cloud. The SwiftShip Tracker platform comprises a Data Layer (Parcel, Delivery, Sender, Receiver), an Automation Layer (Flows, Approval

Processes, Apex Classes, Batch Apex, Email Alerts), an AI Layer (Agentforce AI, Prompt Builder, Parcel Tracking Agent), and a Security Layer (Profiles, Roles, Permission Sets, Sharing Rules, Field-Level Security). Reports and dashboards provide parcel tracking and operational insights.

Figure 7: Solution Architecture



3.2.4 Project Planning

The project followed an Agile methodology with Epics, Stories, and Story Points, sprint-based execution, and velocity calculation for estimation. The product backlog is given below.

Sprint	Epic	USN	User Story / Task	Points	Priority	Team Member
Sprint-1	Developer Setup	USN-1	As a system administrator, I want to create and configure the Salesforce environment so that I can build the SwiftShip Tracker parcel management system.	3	High	Member1
Sprint-2	Data Modeling	USN-2	As an admin, I want to create Parcel, Delivery, Sender, and Receiver objects with proper relationships so that parcel and delivery data is stored and managed systematically.	5	High	Member2
Sprint-2	Data Modeling	USN-3	As a staff member, I want dedicated tabs and a Lightning App for SwiftShip modules so that I can easily navigate and manage parcel and delivery operations.	5	High	Member3
Sprint-3	Automation	USN-4	As a system, I want to validate parcel details, mandatory fields, and booking inputs so that invalid or incomplete parcel records are prevented.	3	High	Member4

Sprint	Epic	USN	User Story / Task	Points	Priority	Team Member
Sprint-3	Automation	USN-5	As a delivery coordinator, I want automated Flows to update parcel status and delivery information so that parcel processing requires minimal manual effort.	3	High	Member4
Sprint-4	Apex	USN-6	As a system, I want Apex triggers and classes to automatically perform critical parcel and delivery actions so that important processes are handled accurately.	5	High	Member1
Sprint-4	Apex	USN-7	As an inventory/operations manager, I want Batch Apex jobs to process and update parcel or delivery records automatically so that operational data remains accurate and up to date.	5	High	Member2
Sprint-5	Reports	USN-8	As a manager, I want reports showing parcel status, delivery performance, and operational metrics so that I can monitor SwiftShip operations and make informed decisions.	4	High	Member3
Sprint-6	Dashboards & AI	USN-9	As a manager, I want dashboards showing parcel status, delivery performance, and key metrics, while Agentforce provides AI-powered tracking assistance, so that I can monitor operations efficiently.	4	Medium	Member4

3.2.5 Data Model and ER Diagram

The data model consists of four custom objects. Parcel__c stores core parcel details and is linked to Sender__c; Delivery__c tracks the real-time location and estimated delivery date and is linked to Parcel__c and Sender__c; Sender__c captures sender contact information; and Receiver__c stores receiver details and is linked to Sender__c and Parcel__c.

Parcel Object (Parcel__c): stores parcel information

Field Name	API Name	Data Type	Description
Parcel ID	Parcel_ID__c	Auto number	Unique ID for each parcel
Status	Status__c	Picklist	Booked, In Transit, Out for Delivery, Delivered
Weight	Weight__c	Number	Weight of parcel
Estimated Delivery	Estimated_Delivery_Date__c	Date	Expected delivery date
Sender	Sender__c	Lookup Relationship	Gets details from Sender

Delivery Object (Delivery__c)

Field Name	API Name	Data Type	Description

Field Name	API Name	Data Type	Description
Current Location	Current_Location__c	Geolocation	Location in terms of longitude and latitude
Estimated Delivery	Estimated_Delivery_Date__c	Date	Expected delivery date
Sender	Sender__c	Lookup Relationship	Gets details from Sender
Parcel	Parcel__c	Lookup Relationship	Gets details from Parcel

Sender Object (Sender__c)

Field Name	API Name	Data Type	Description
Sender Address	Sender_Adress__c	Geolocation	Location in terms of longitude and latitude
Sender Contact	Sende_Contact__c	Phone	To contact the sender
Sender Email	Sender_Email__c	Email	To send email notification

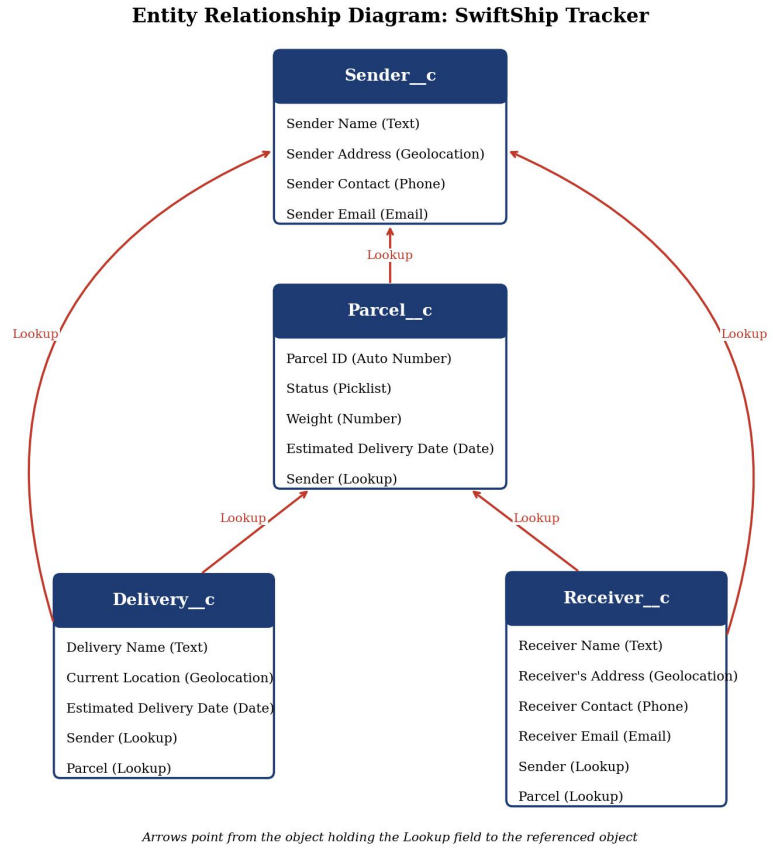
Receiver Object (Receiver__c)

Field Name	API Name	Data Type	Description
Receiver Address	Receiver_Adress__c	Geolocation	Location in terms of longitude and latitude
Receiver Contact	Receiver_Contact__c	Phone	To contact the receiver
Receiver Email	Receiver_Email__c	Email	To send email notification
Sender	Sender__c	Lookup Relationship	Gets details from Sender
Parcel	Parcel__c	Lookup Relationship	Gets details from Parcel

Object Relationships

Child Object	Lookup Field	Parent Object
Parcel__c	Sender__c	Sender__c
Delivery__c	Sender__c	Sender__c
Delivery__c	Parcel__c	Parcel__c
Receiver__c	Sender__c	Sender__c
Receiver__c	Parcel__c	Parcel__c

Figure 8: Entity Relationship (ER) Diagram



3.2.6 Security Model Design

- **Role Hierarchy:** Administrator → Delivery Manager → Agent → Customer.
- **Record-Level Security:** Sharing Rules are applied for Parcel__c and Delivery__c to allow controlled access for agents and managers. Customers can only view records linked to their own Sender or Receiver details.
- **Field-Level Security:** Sensitive fields such as Sender Email, Receiver Contact, and Location Coordinates are restricted to Admin and Agent profiles only.

Profile	Access
Admin Profile	Full access to all objects and records
Delivery Agent Profile	Read/Edit access to Parcel, Delivery, Sender, and Receiver objects
Customer Profile	Read-only access to their own parcel details via Experience Cloud
Support Staff Profile	Access to Cases and Parcel details for issue resolution

Permission Sets are used to grant additional privileges, for example “Update Delivery Status” and “Access Agent Console”. The permission set implemented in this project is described in Phase 3.

3.3 Phase 2: Salesforce Development: Backend & Configurations

This phase focused on developing the core logic of SwiftShip Tracker. Custom objects were created for Parcel, Delivery, Sender, and Receiver with relationships. A Prompt Builder template and an Auto-launched Flow were then built to retrieve parcel details for the AI agent. The phase is organized into six milestones.

3.3.1 Milestone 1: Salesforce Developer Account

Salesforce is a customer success platform designed to help organizations sell, service, market, analyze, and connect with their customers. A Salesforce Developer Org was created to configure, develop, and test the project.

Creating the Developer Account

1. Go to https://developer.salesforce.com/signup.
2. On the sign-up form, enter First name and Last name, Email, Job Title (Developer), Company (College Name), and Country (India), then click Sign Me Up.
3. Open the inbox of the email used for sign-up and click Verify Account to activate the account. The email may take 10 to 30 minutes, and sometimes up to 2 hours, to arrive.
4. Set a password, answer a security question, and click Change Password.
5. The user is then redirected to the Salesforce Setup page.

Figure 9: Salesforce Developer Edition Sign-Up Page

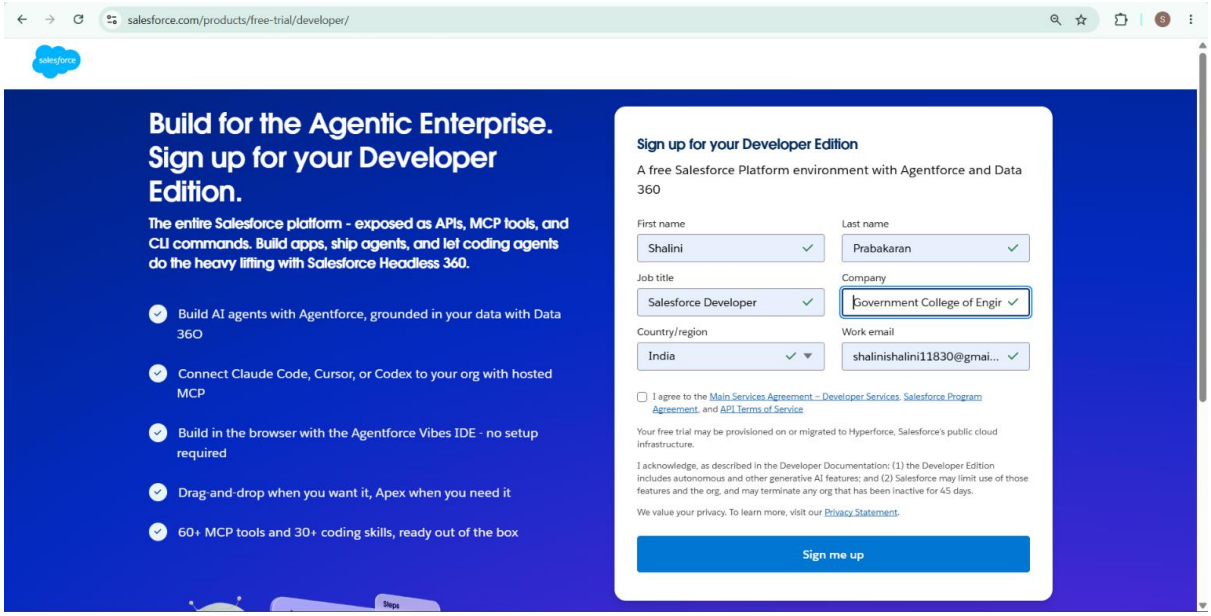


Figure 10: Account Verification Email

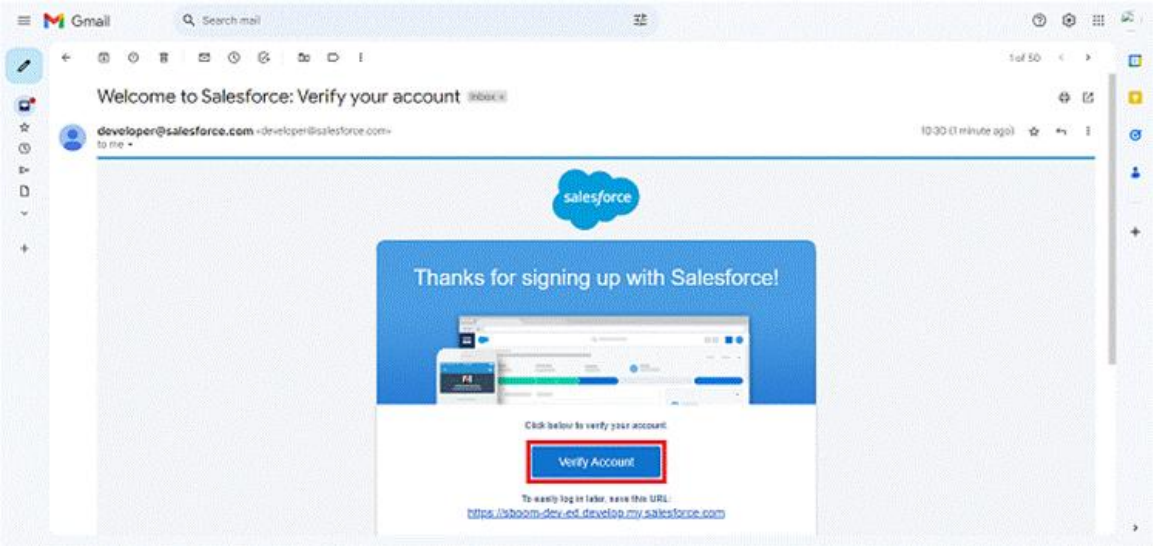


Figure 11: Change Password Page

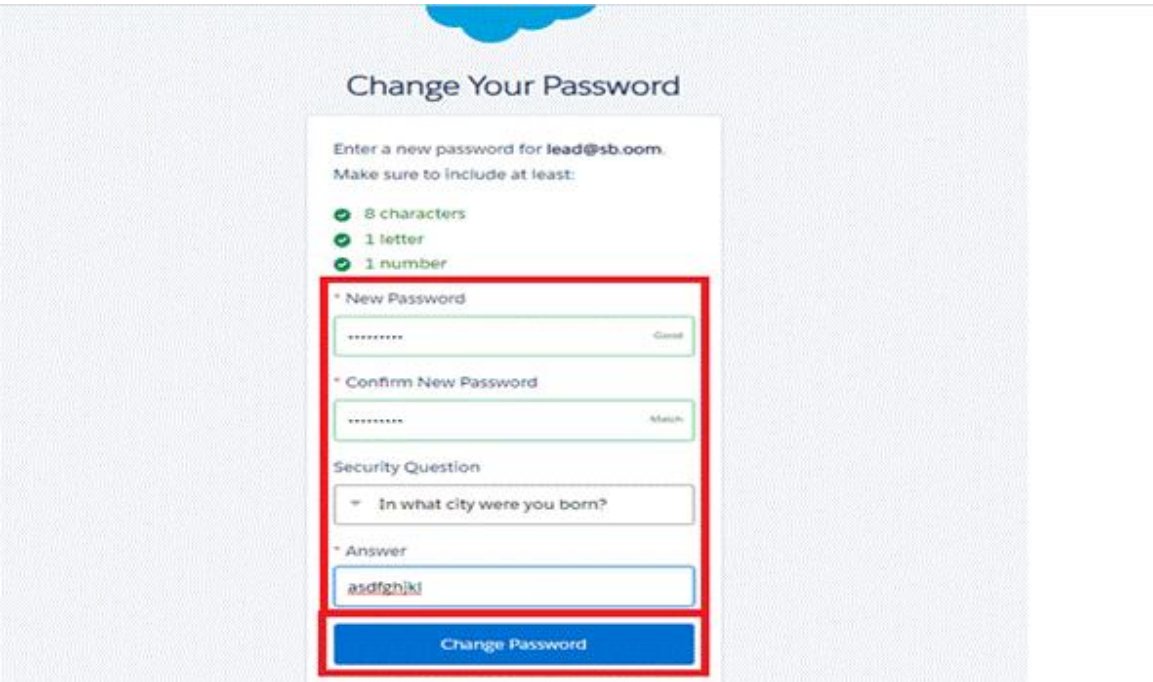
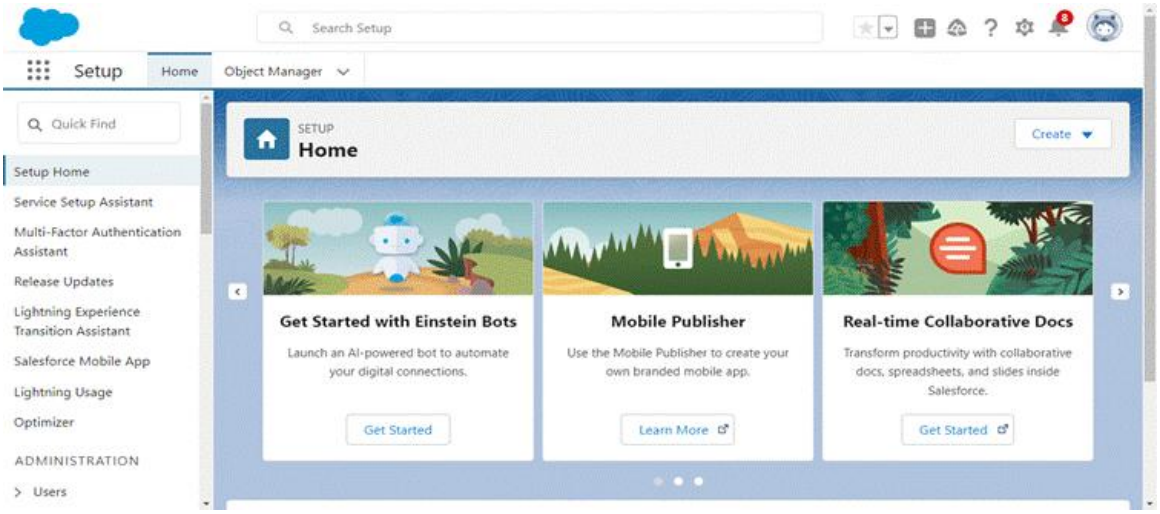


Figure 12: Salesforce Setup Home Page



3.3.2 Milestone 2: Objects Creation

Four custom objects were created: Parcel, Delivery, Sender, and Receiver. The same procedure was followed for each object.

1. From the Setup page, click Object Manager.
2. Click Create, then click Custom Object.
3. Enter the object label and plural label, and enter the Record Name and select Data Type Text.
4. Select Allow Reports, Allow Search, and Track Field History.
5. Click Save (or Save & New to create the next object).

Object	Label	Record Name	API Name
Parcel	Parcel	Parcel Name	Parcel__c
Delivery	Delivery	DeliveryName	Delivery__c
Sender	Sender	SenderName	Sender__c
Receiver	Receiver	ReceiverName	Receiver__c

Figure 13: Parcel Object: New Custom Object Configuration

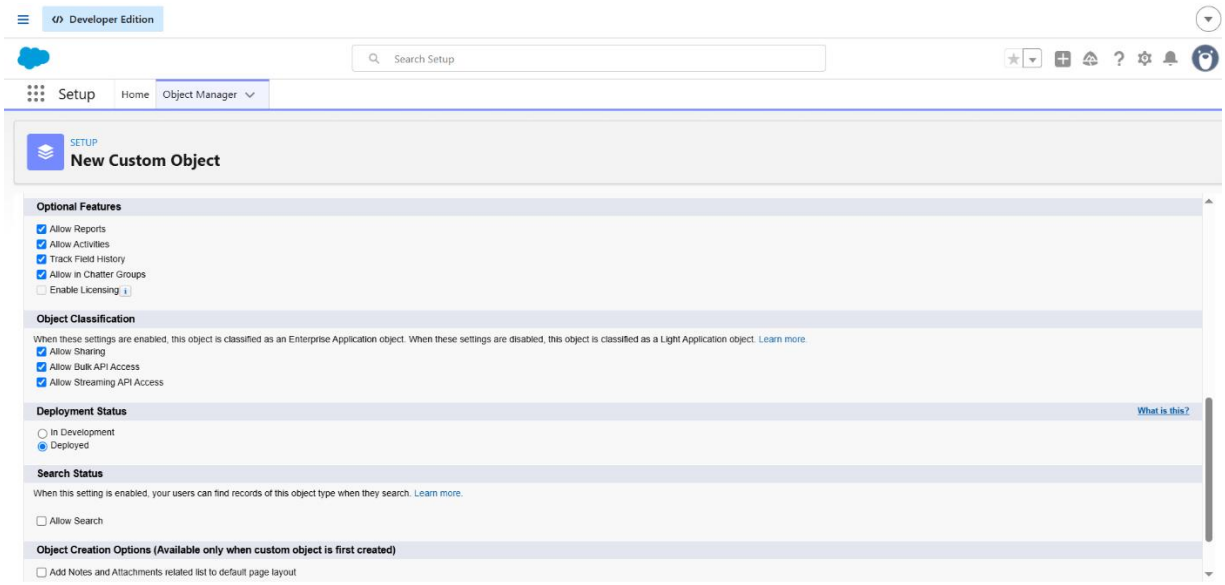


Figure 14: Parcel Object Details in Object Manager

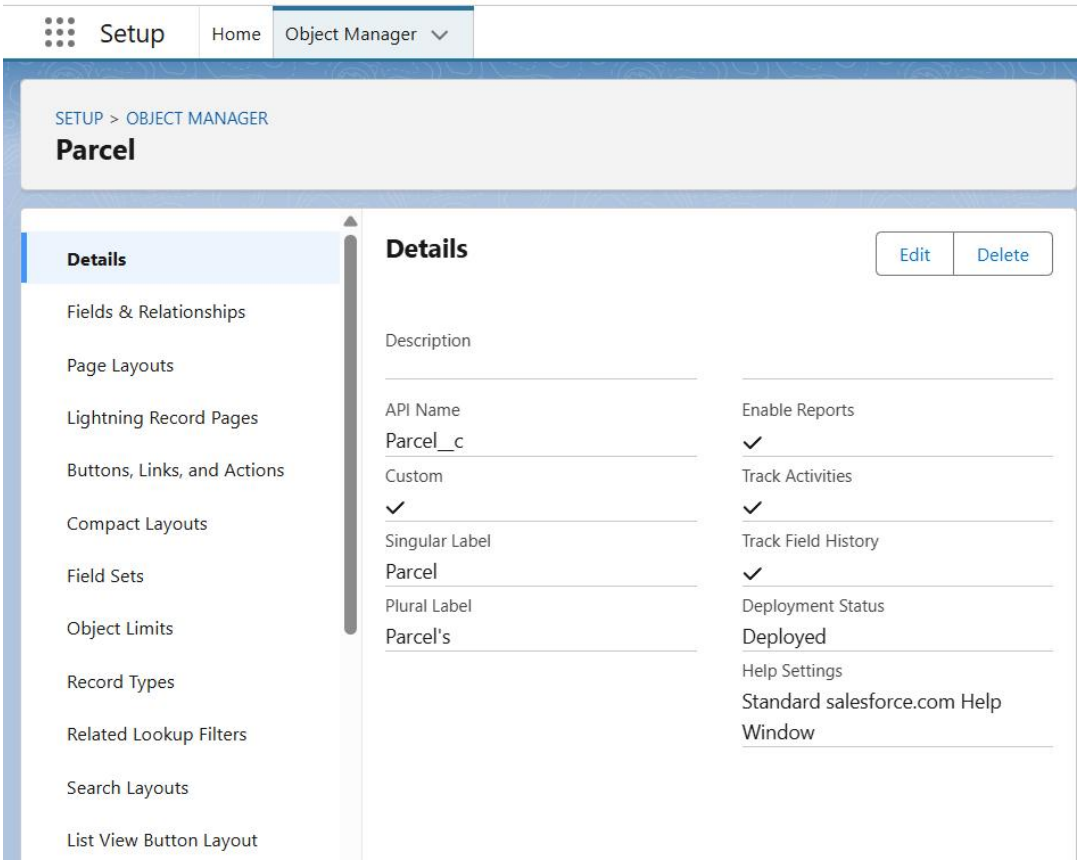


Figure 15: Delivery Object: New Custom Object Configuration

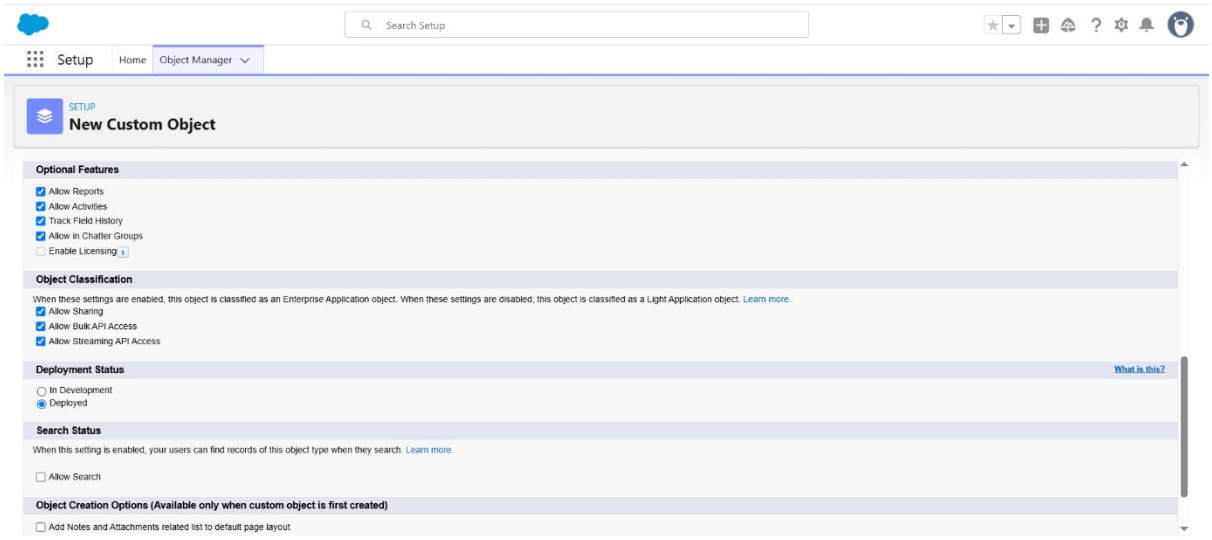


Figure 16: Delivery Object Details in Object Manager

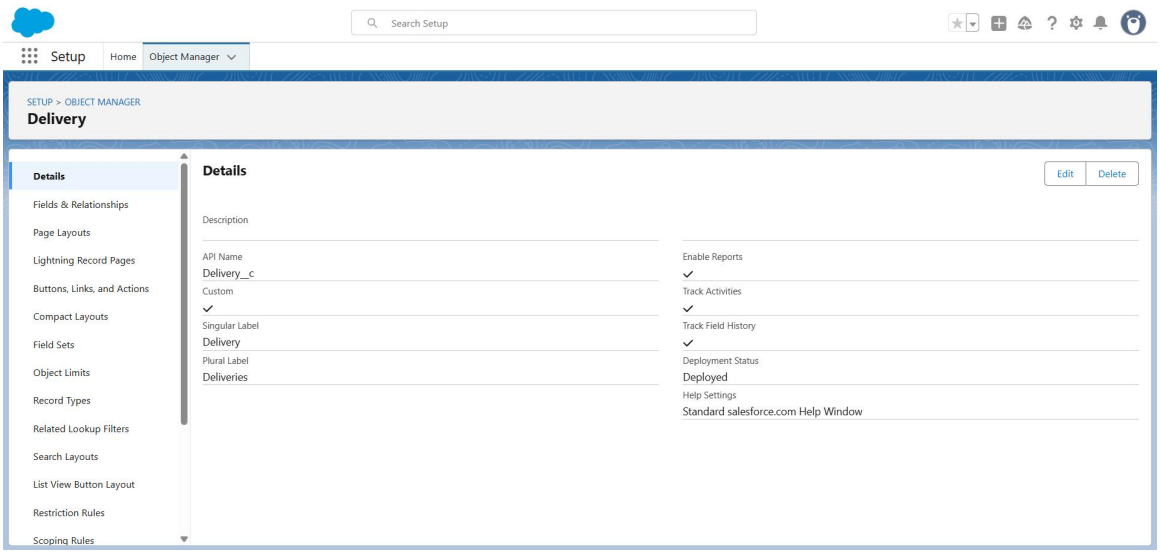


Figure 17: Sender Object: New Custom Object Configuration

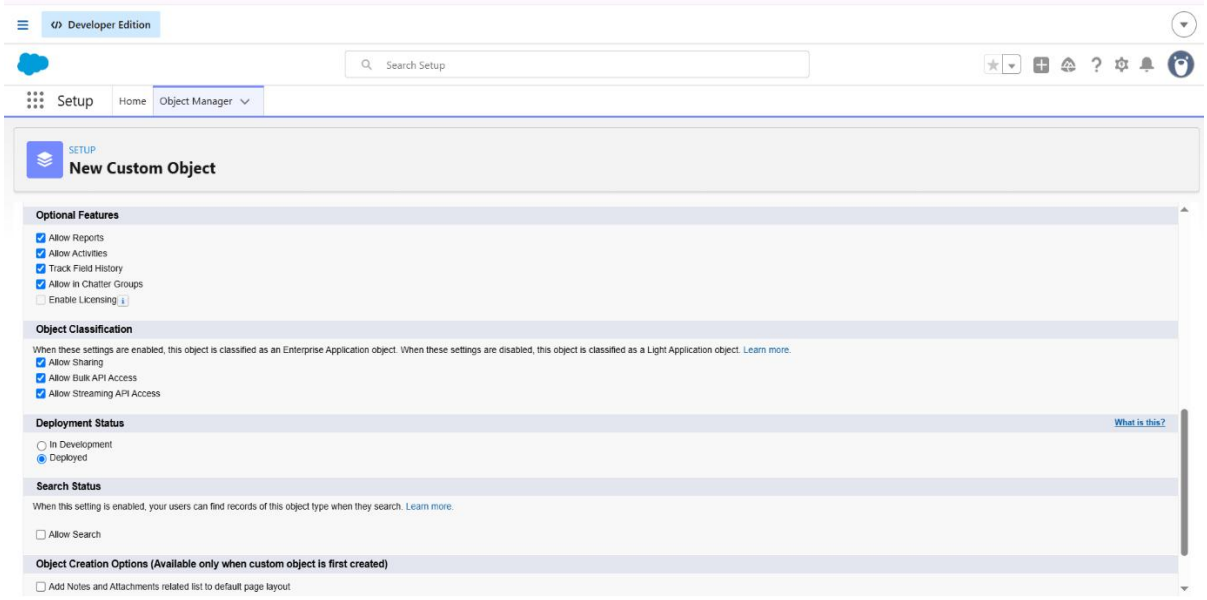


Figure 18: Sender Object Details in Object Manager

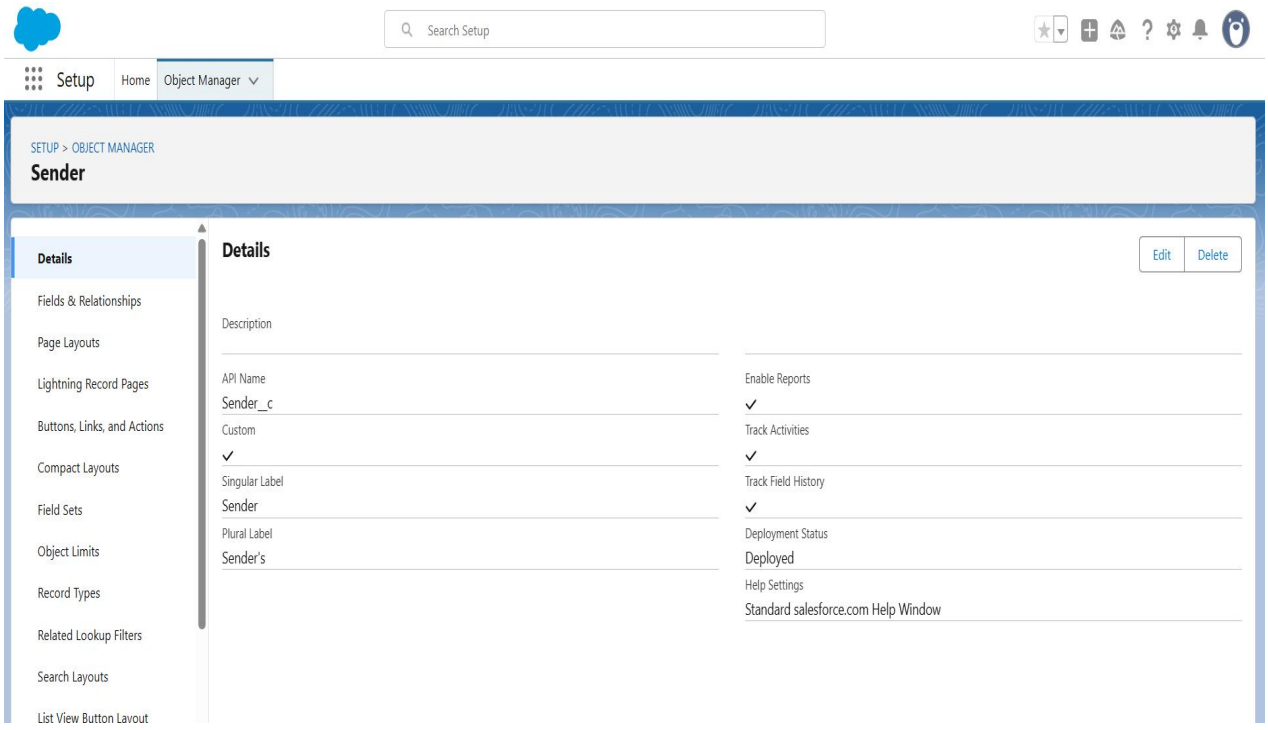
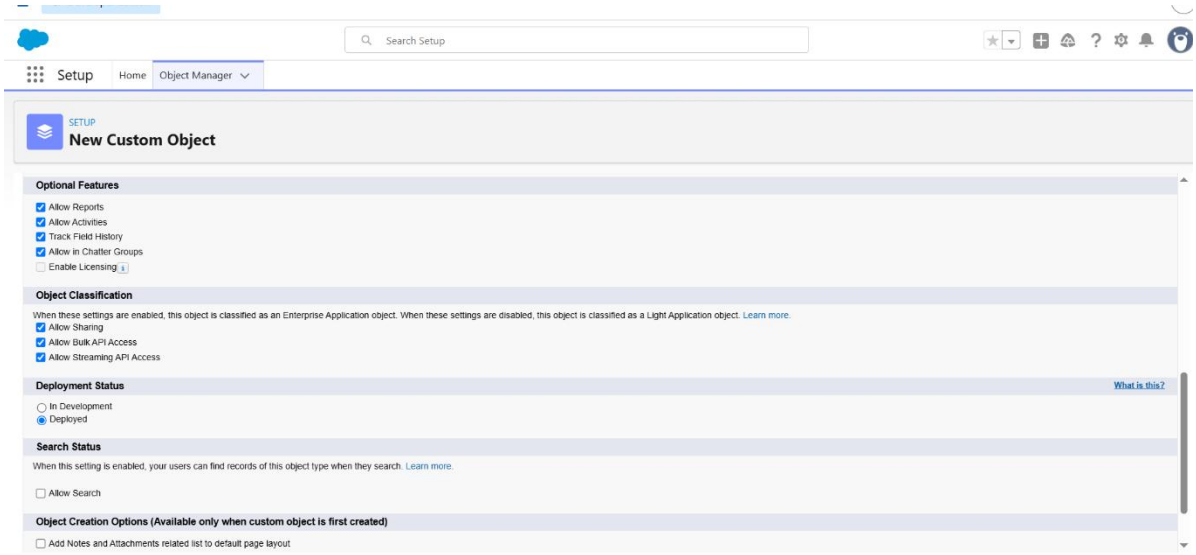


Figure 19: Receiver Object: New Custom Object Configuration



3.3.3 Milestone 3: Tabs

Custom tabs were created for the Parcel, Delivery, Sender, and Receiver objects so that each object can be accessed from the application.

1. Go to Setup and type Tabs in the Quick Find box, then click Tabs.
2. Click New under Custom Object Tabs.
3. Select the object (Parcel, Delivery, Sender, or Receiver) and choose a Tab Style.
4. Click Next and keep the Add to Profiles settings as default, then click Next.
5. On the Add to Custom Apps page, uncheck Include Tab.
6. Ensure Append tab to the user's existing personal customizations is checked, then click Save.
7. Repeat the steps for each object.

Figure 20: Custom Object Tabs: Creating a New Tab

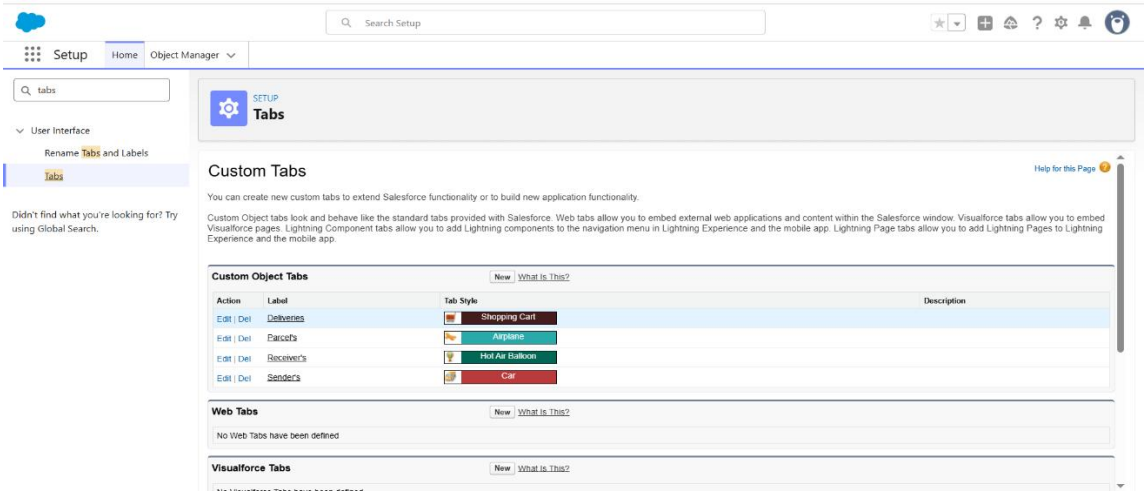


Figure 21: Custom Object Tab Definition for the Parcel Object

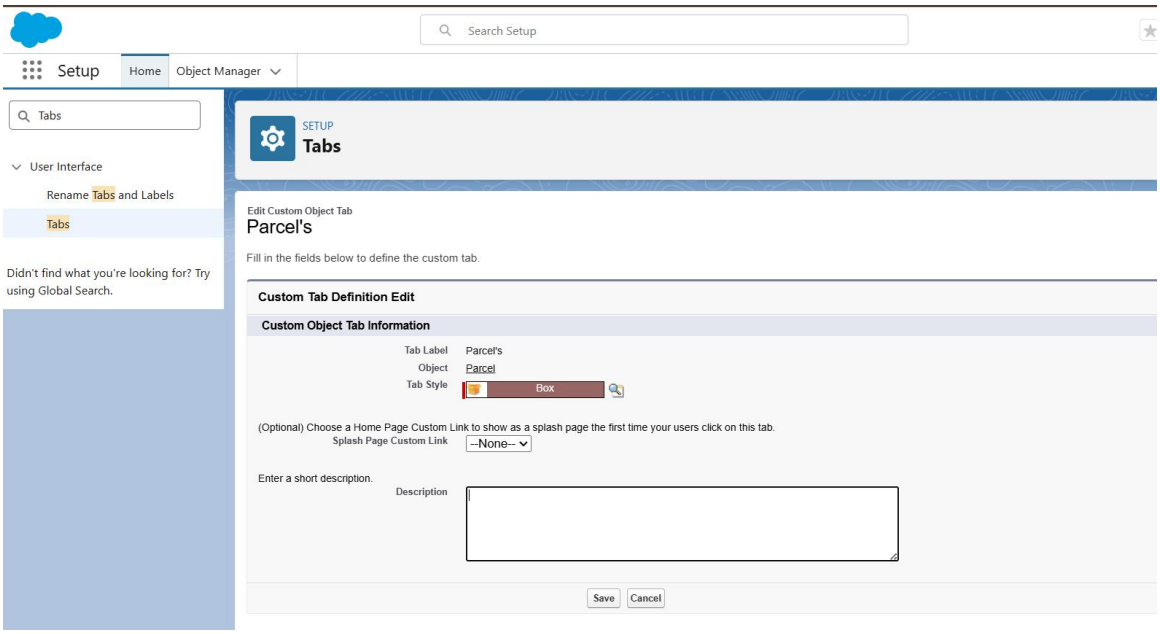
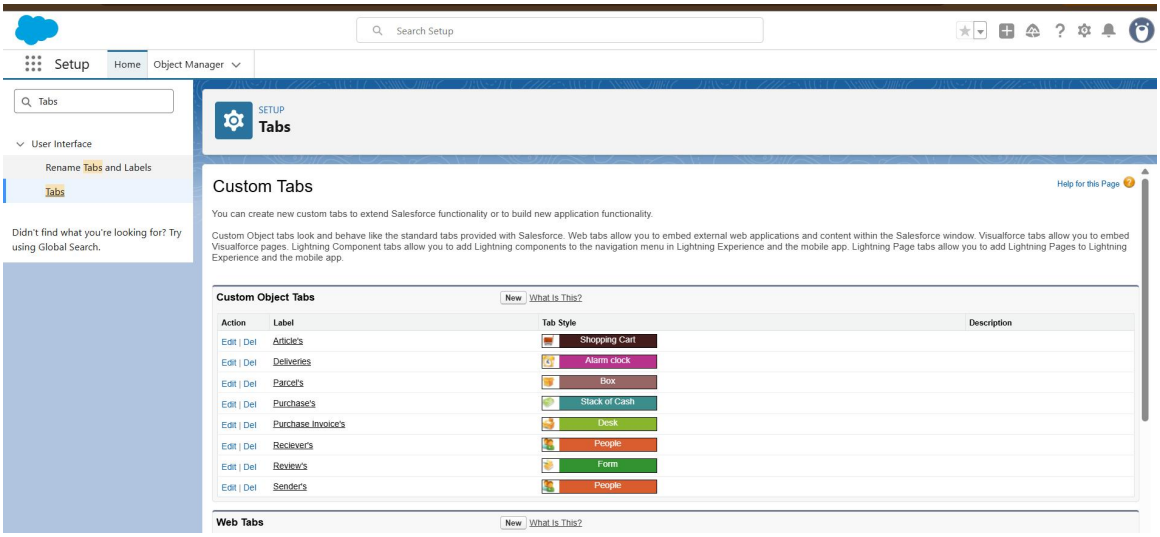


Figure 22: List of Custom Object Tabs



3.3.4 Milestone 4: Fields & Relationships

Fields and relationships were created for each custom object using Object Manager, Fields & Relationships, New. The field definitions are given in Section 3.2.5. The general procedure for each field is:

1. Open Object Manager and select the object.
2. Go to Fields & Relationships and click New.

3. Choose the data type (for example Auto Number, Picklist, Number, Date, Geolocation, Phone, Email, or Lookup Relationship) and click Next.
4. Enter the field label and details, then click Next, Next, and Save & New.

Parcel Object Fields

Parcel ID was created as an Auto Number field with the display format P-{000}. The remaining fields (Status, Weight, Estimated Delivery, Sender) were created using the same procedure.

Figure 23: Parcel Object: Fields & Relationships

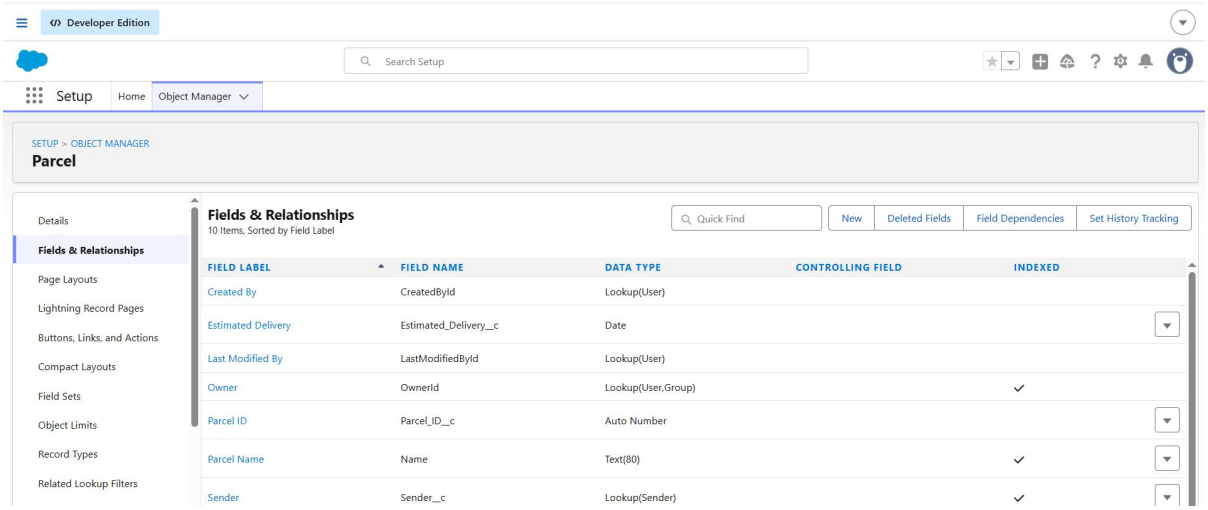
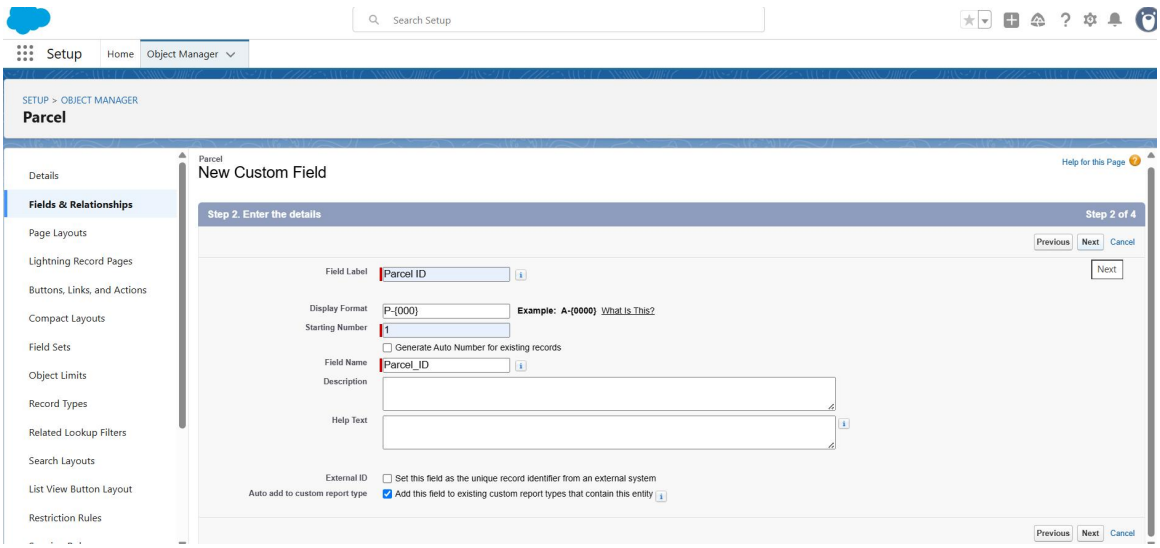


Figure 24: Parcel Object: Creating the Parcel ID Auto Number Field



Delivery Object Fields

Current Location was created as a Geolocation field. The remaining fields (Estimated Delivery, Sender, Parcel) were created using the same procedure.

Figure 25: Delivery Object: Fields & Relationships

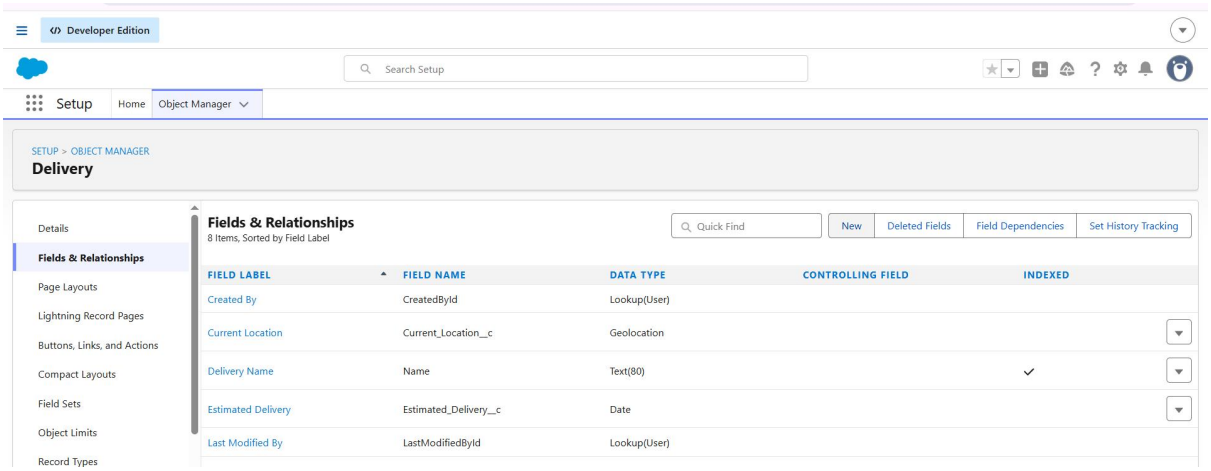
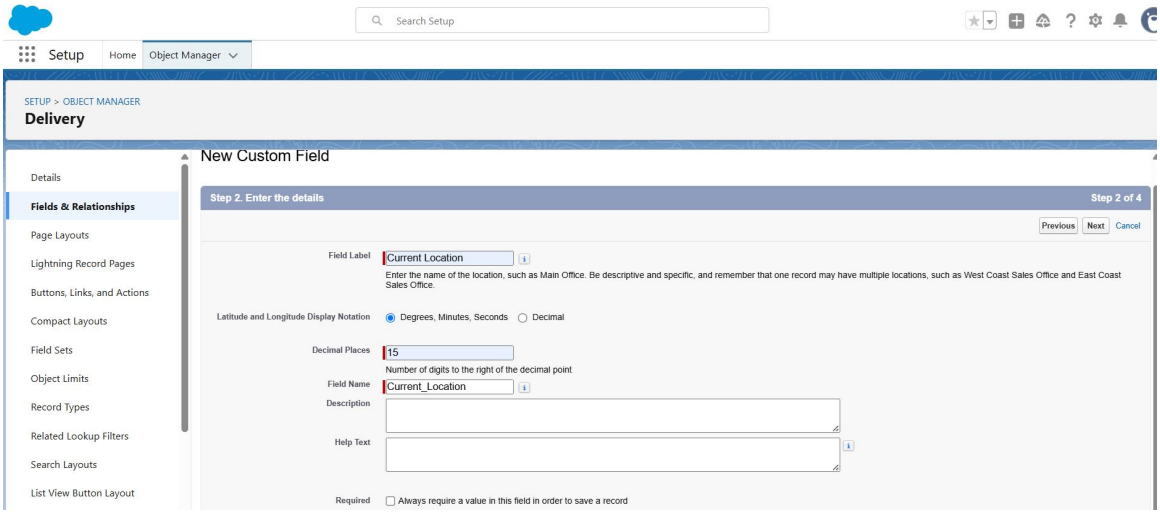


Figure 26: Delivery Object: Creating the Current Location Geolocation Field



Sender Object Fields

Sender Address was created as a Geolocation field. The remaining fields (Sender Contact, Sender Email) were created using the same procedure.

Figure 27: Sender Object: Fields & Relationships

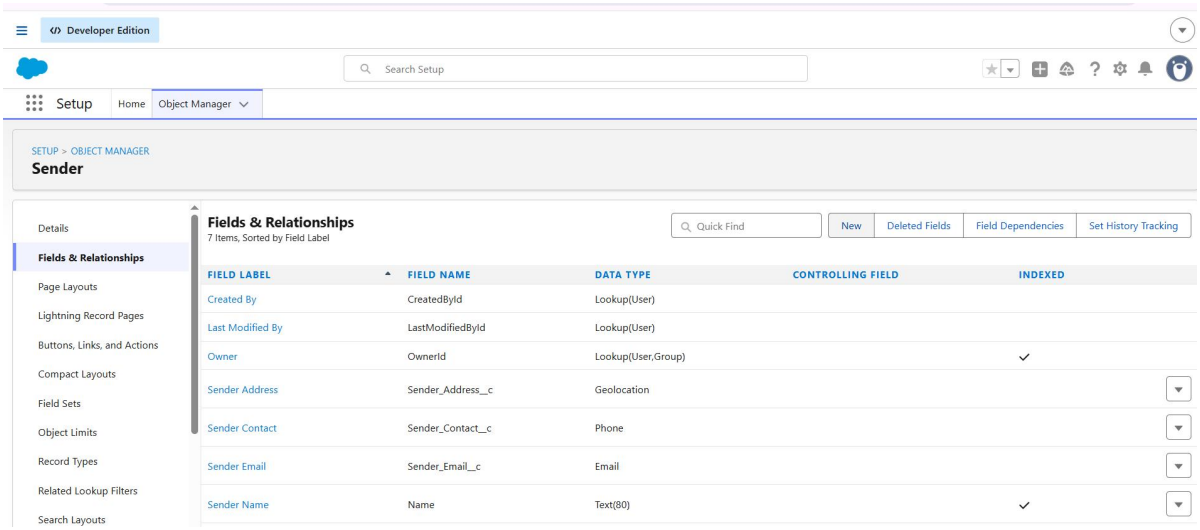
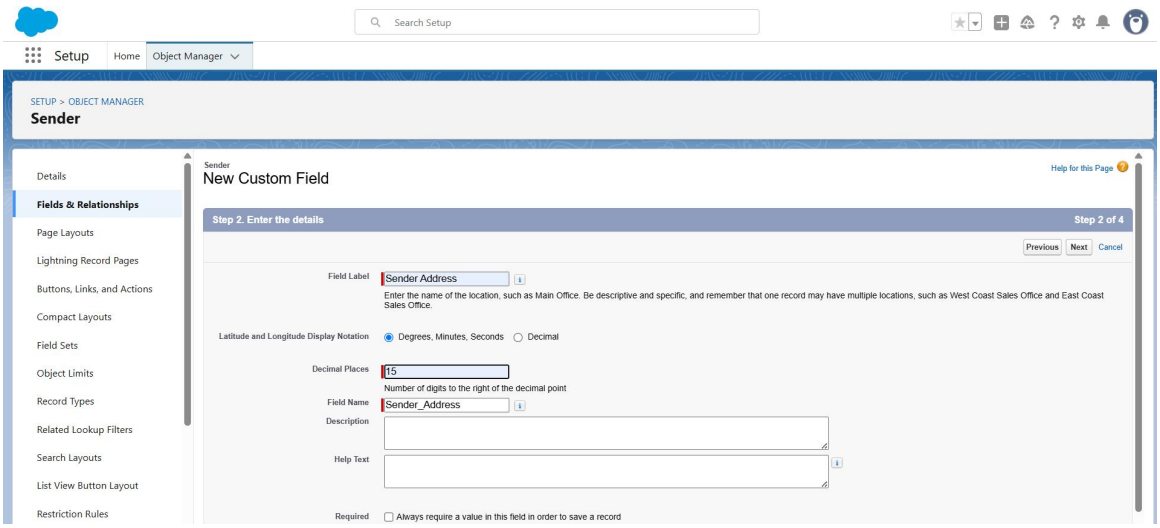


Figure 28: Sender Object: Creating the Sender Address Geolocation Field



Receiver Object Fields

Receiver's Address was created as a Geolocation field. The remaining fields (Receiver Contact, Receiver Email, Sender, Parcel) were created using the same procedure.

Figure 29: Receiver Object: Fields & Relationships

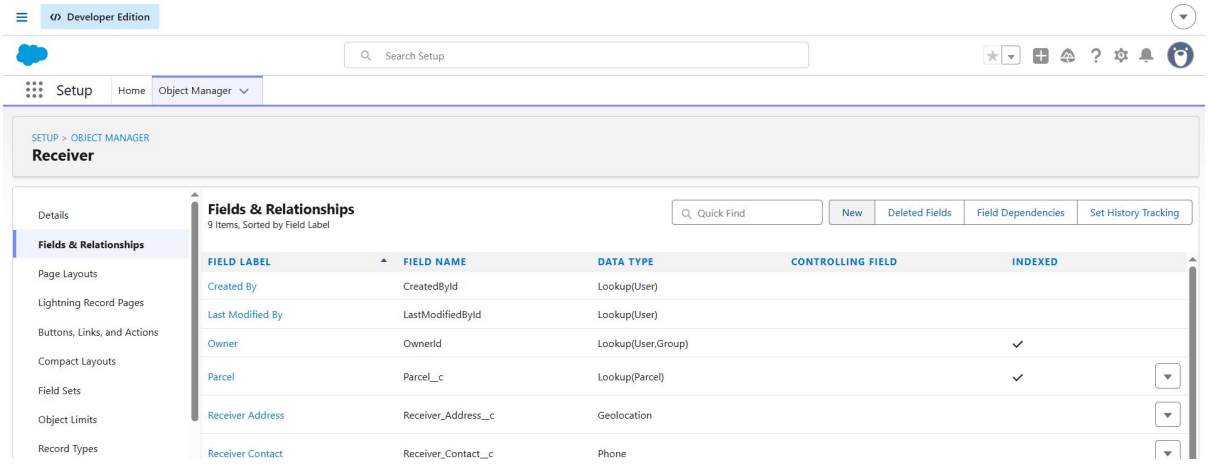
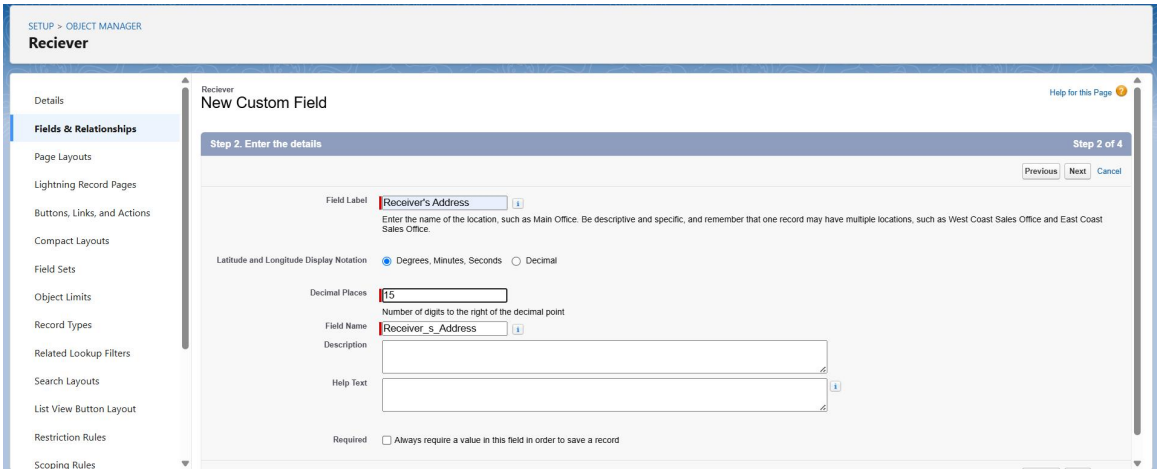


Figure 30: Receiver Object: Creating the Receiver's Address Geolocation Field



3.3.5 Milestone 5: Prompt Builder

A Prompt Template was designed in Prompt Builder to retrieve parcel details for the Agentforce agent.

1. In Setup, type Prompt Builder in the Quick Find box and open it.
2. Click New Prompt Template and select the Record Summary template type, with Parcel as the object type. Click Start from Scratch.
3. Enter the details: Label "Retrieve Parcel Details", API Name "Retrieve_Parcel_Details", and Description "Retrieves parcel details where Parcel_ID matches the user input."
4. Enter the prompt template content (given below) in the Prompt Template text area.

5. Use Insert Resource to map the Parcel__c fields: Name, Parcel_ID__c, Status__c, Weight__c, and Estimated_Delivery_Date__c.
6. Click Save, then click Activate to make the prompt template available for the agent or Flow.

Prompt Template Content

Retrieve details from Parcel__c where Parcel_ID__c matches user input. Return response in this format:

Parcel Tracking Update

- Parcel Name: {!$Input:Parcel__c.Name}

- Parcel ID: {!$Input:Parcel__c.Parcel_ID__c}

- Status: {!$Input:Parcel__c.Status__c}

- Weight: {!$Input:Parcel__c.Weight__c}

- Estimated Delivery Date: {!$Input:Parcel__c.Estimated_Delivery_Date__c}

Figure 31: Prompt Builder: Creating a New Prompt Template

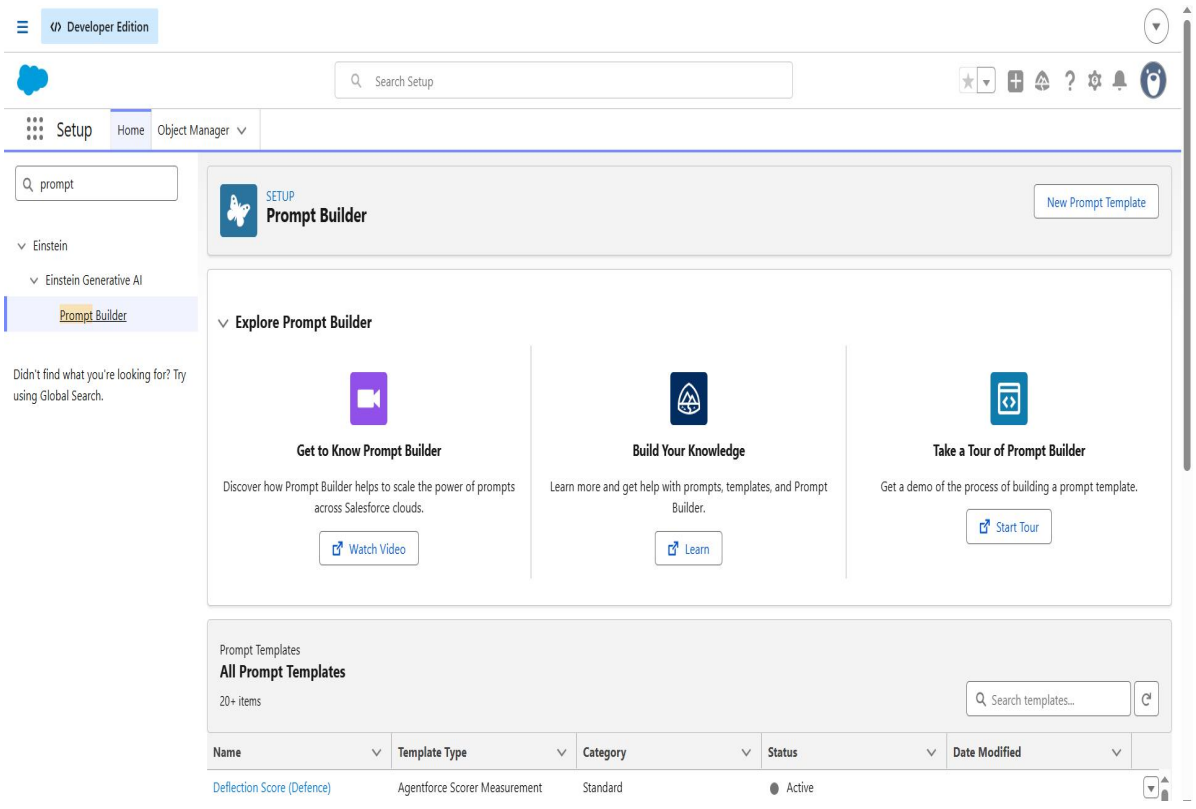


Figure 32: Prompt Template Type and Object Selection (Record Summary: Parcel)

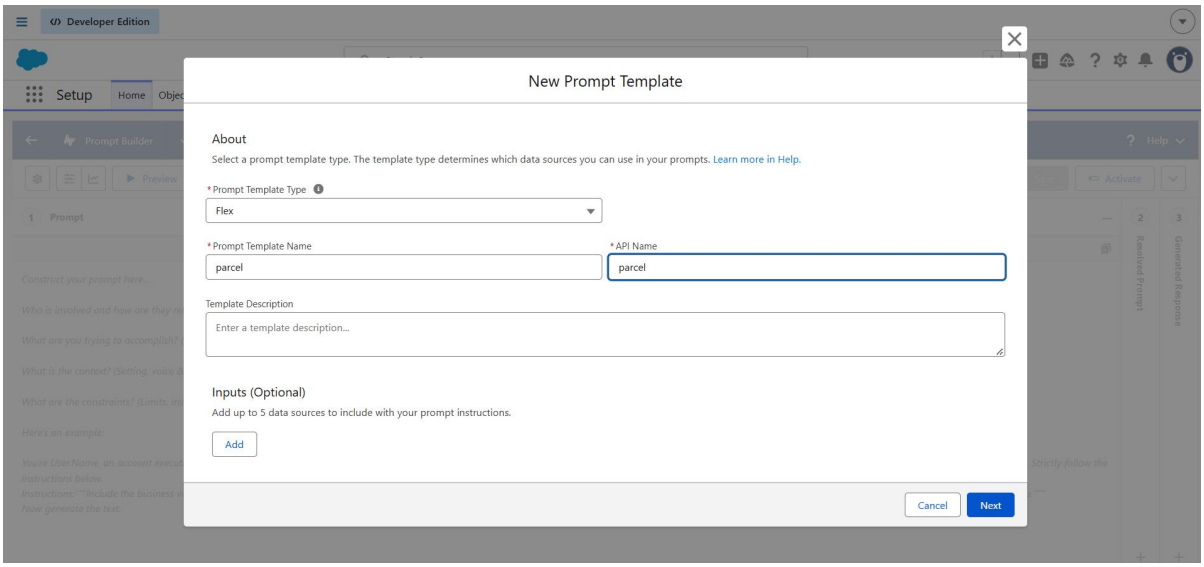


Figure 33: Prompt Template Content and Resource Mapping

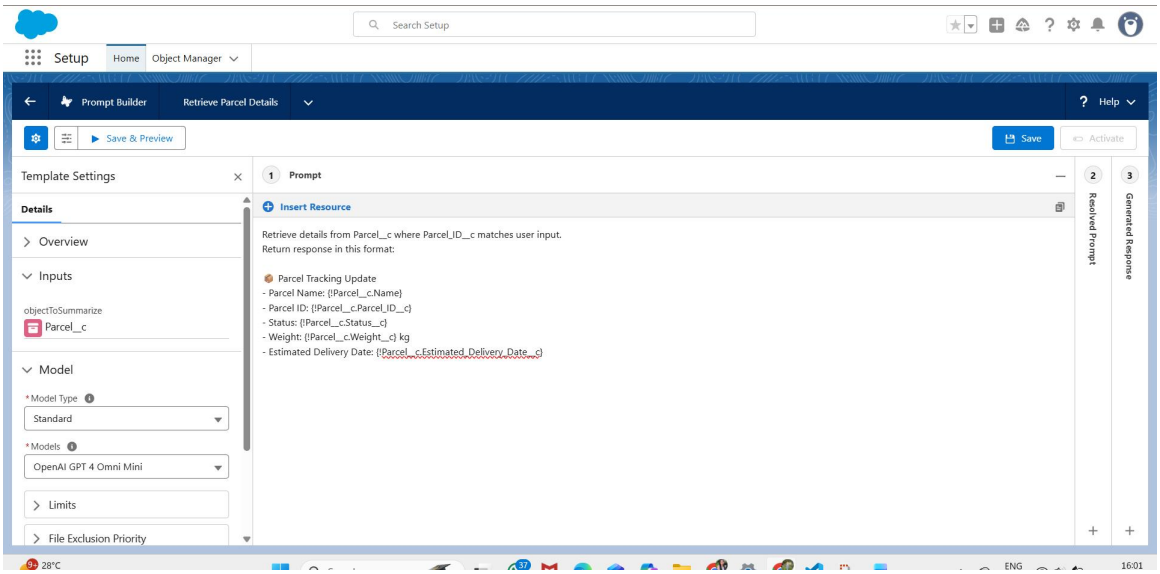
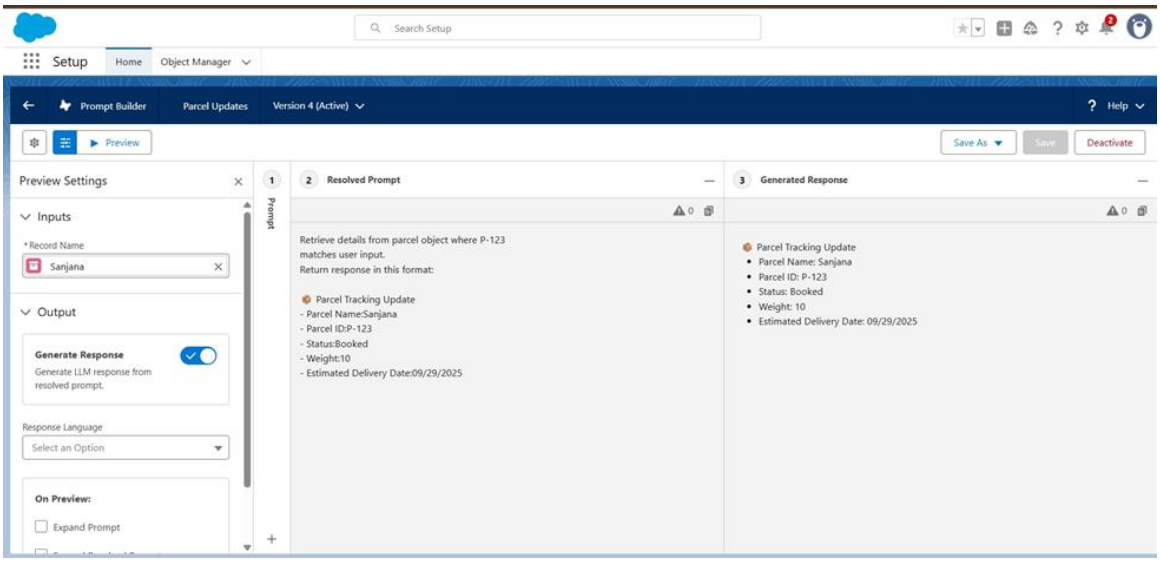


Figure 34: Prompt Template Preview and Generated Response



3.3.6 Milestone 6: Flows

An Auto-launched Flow was created to capture the prompt response for the agent.

1. Go to Setup, search for Flows, click New Flow, and choose Autolaunched Flow (No Trigger).
2. Add input and output variables: “ids” (Text, Available for Input) and “output” (Text, Available for Output).
3. Add a Get Records element labeled “Get Parcel Records” with Object = Parcel, Field = Parcel_ID__c, Operator = Equals, and Value = ids.
4. Add an Action element for the Retrieve Parcel Details prompt template, labeled “Parcel Updates”, with Object To Summarize set to the Parcel record from Get Parcel Records.
5. Add an Assignment element labeled “Assignment Outputs”: Variable = output, Operator = Equals, Value = Outputs from Action > Prompt Response.
6. Connect Start, Get Records, Action, Assignment, and End.
7. Save the Flow with the name “Parcel Details” and activate it.
8. From the Flows list, open the drop-down for the new Flow, click Edit Access, and assign the required profiles under Flow user access, then click Save.

Figure 35: Flow Builder: Selecting Autolaunched Flow (No Trigger)

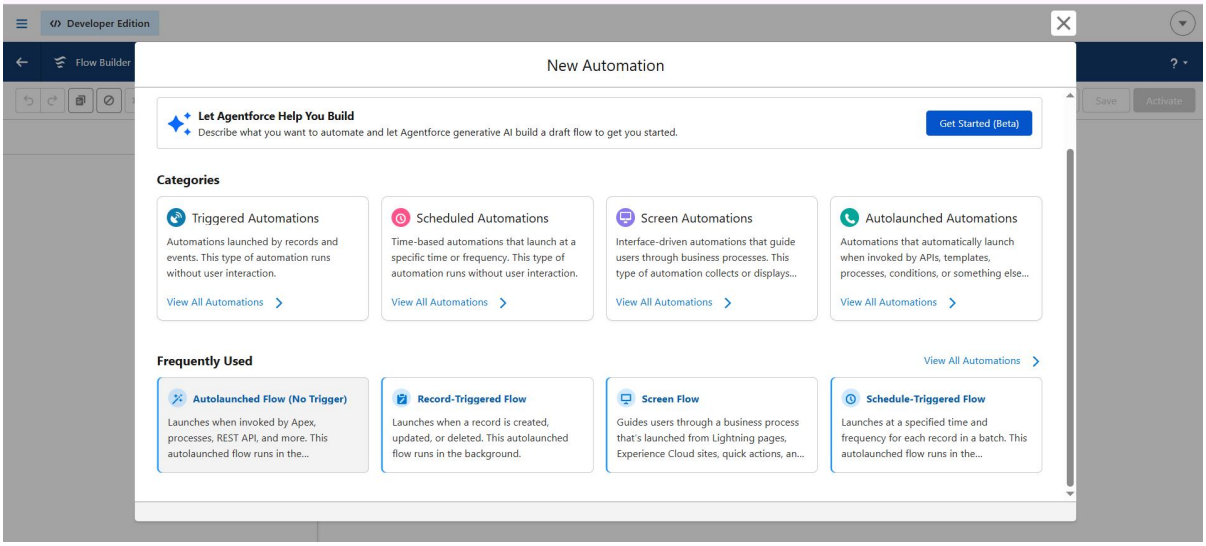


Figure 36: Flow Builder: Creating a New Variable Resource

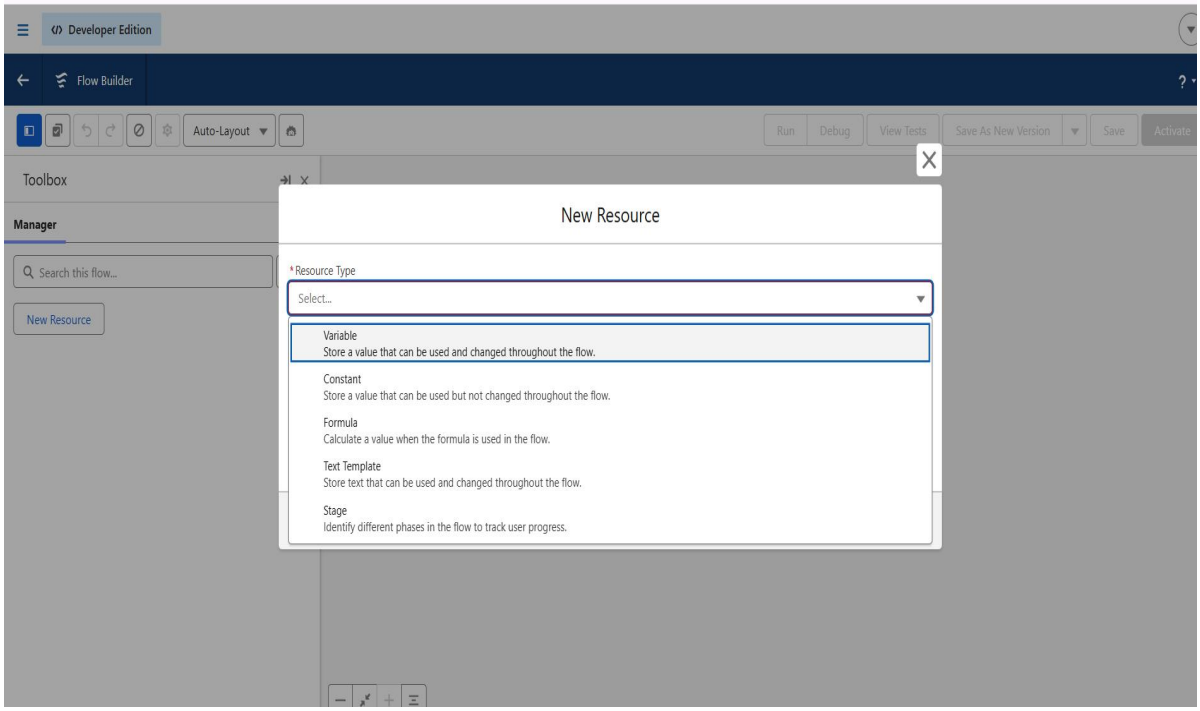


Figure 37: Flow Variable: ids (Available for Input)

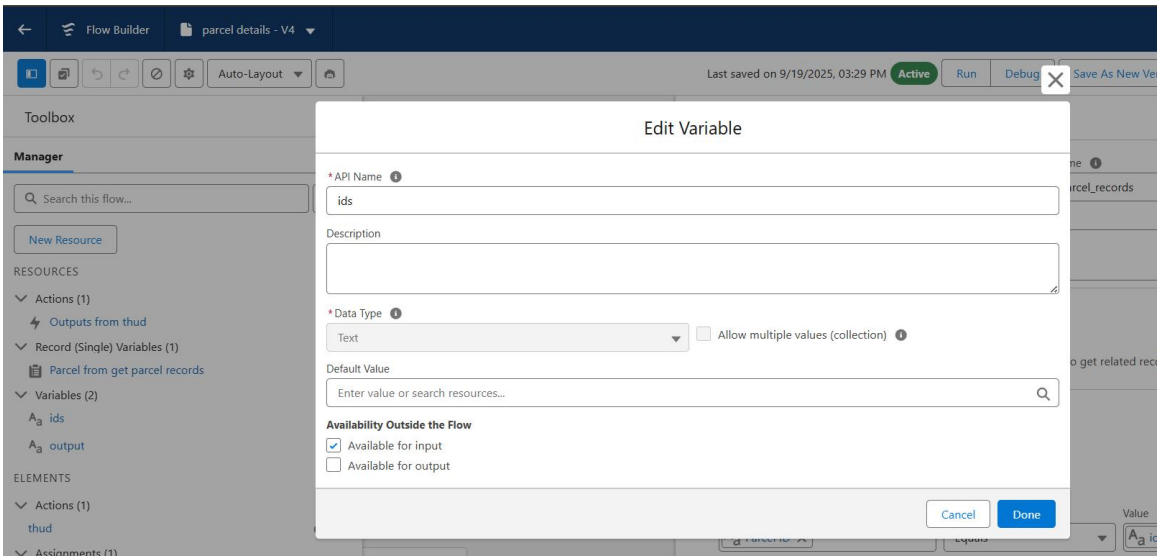


Figure 38: Flow Variable: output (Available for Output)

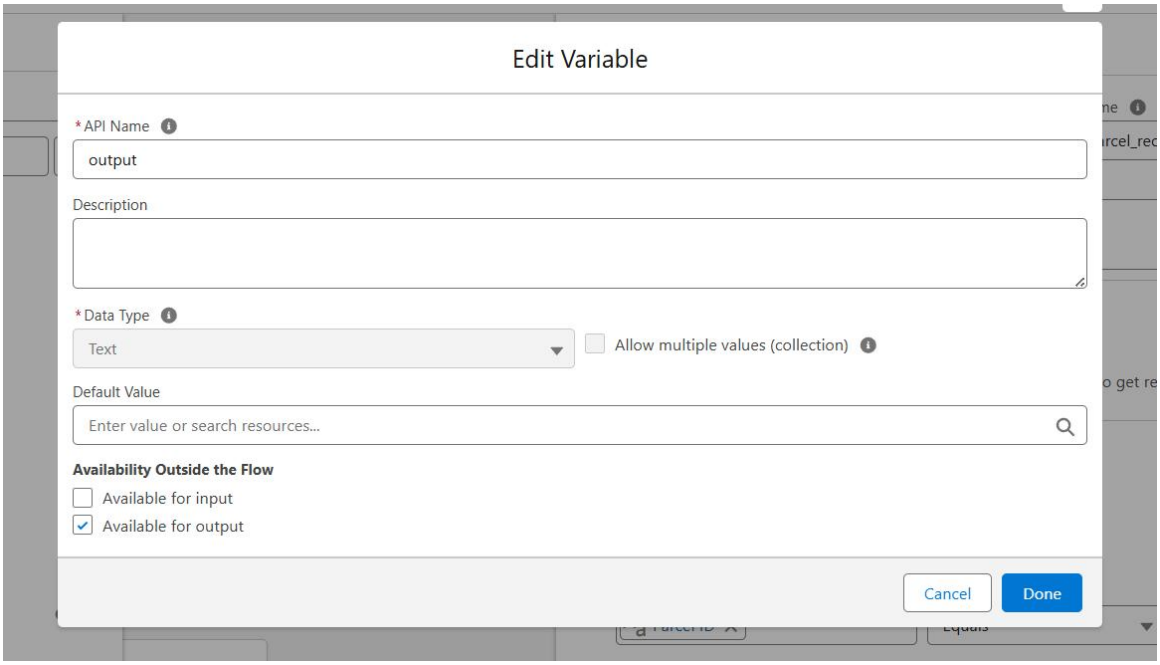


Figure 39: Flow Elements and Get Records Configuration

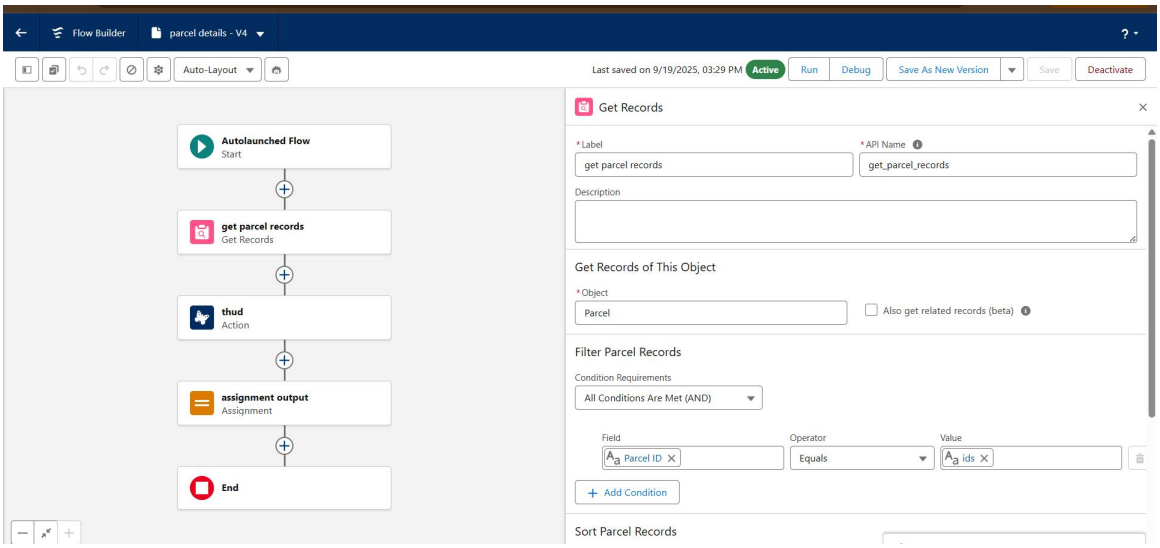


Figure 40: Flow Action Configuration: Parcel Updates

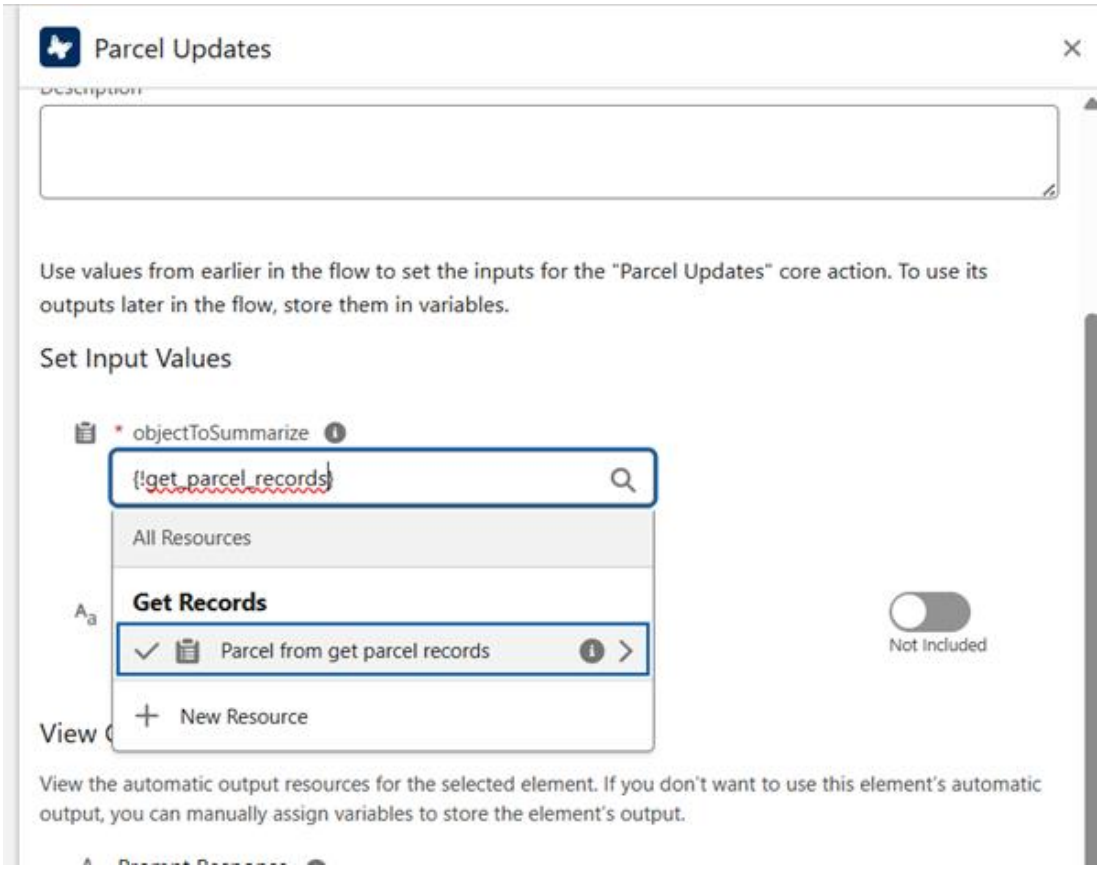


Figure 41: Flow Assignment Element: Assignment Outputs

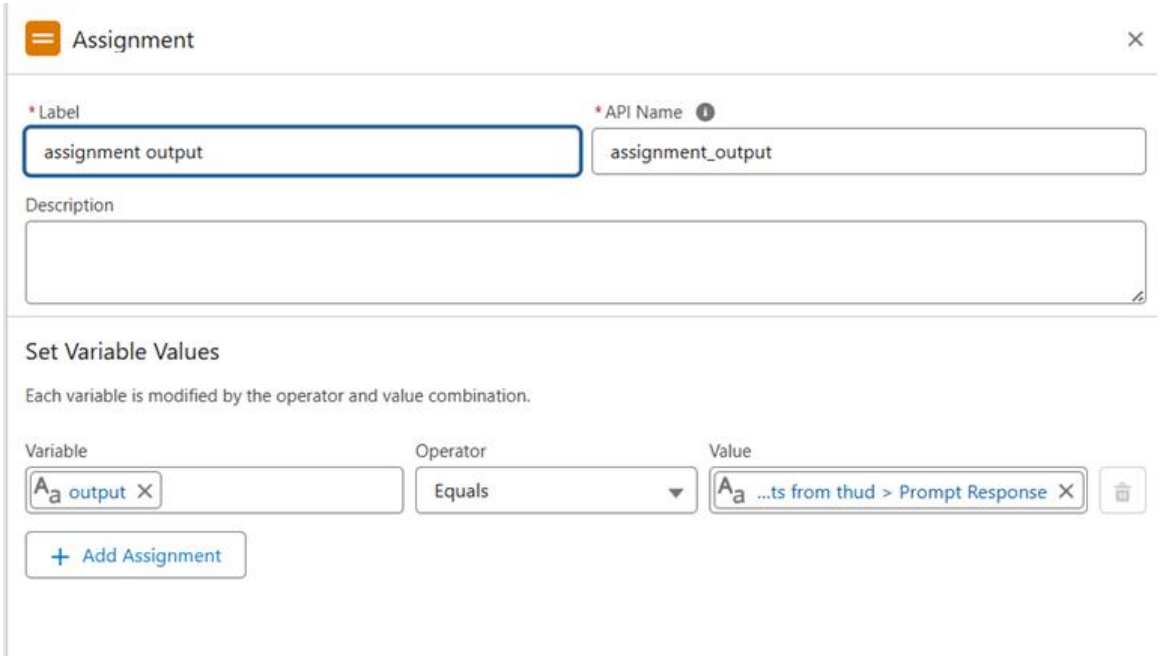


Figure 42: Flows List: Edit Access Option for the Parcel Details Flow

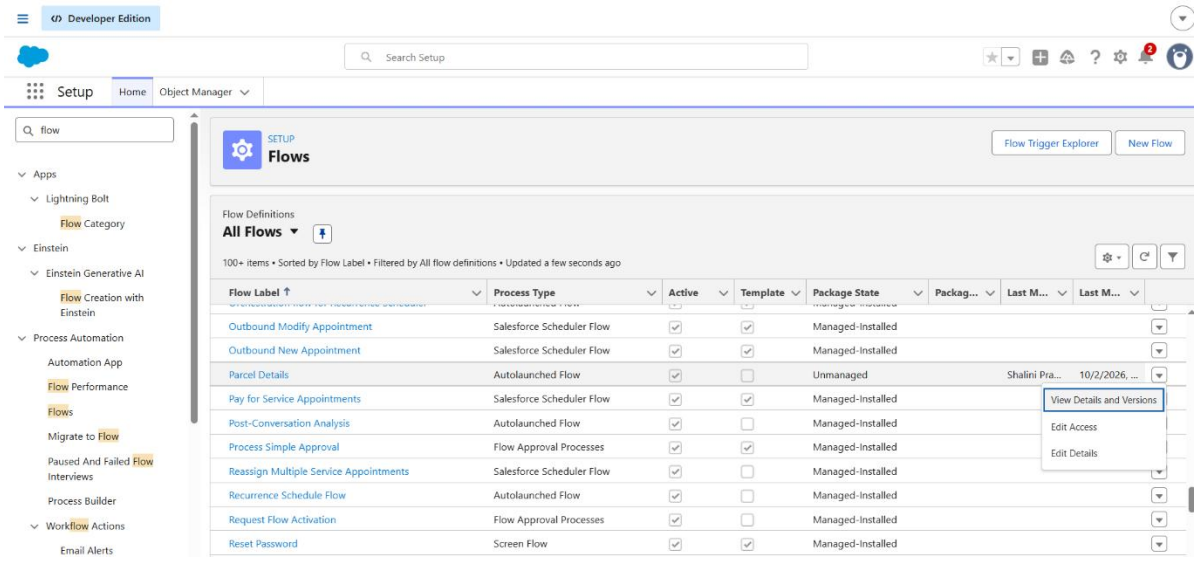
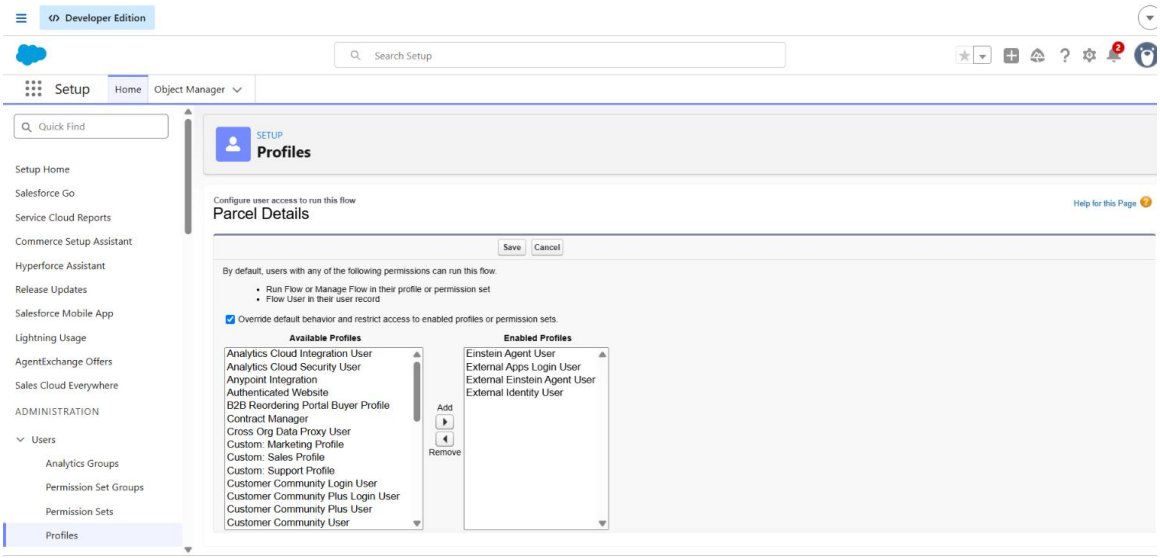


Figure 43: Flow User Access: Enabled Profiles



3.3.7 Automation Components Summary

The table below lists the automation components defined for SwiftShip Tracker and indicates which of them are demonstrated in the screenshots of this documentation.

Component	Purpose	Status in this documentation
Auto-launched Flow (Parcel Details)	Retrieves the parcel record by Parcel ID and returns the prompt response to the agent	Implemented (Milestone 6)
Prompt Template (Retrieve Parcel Details)	Formats Parcel ID, Status, Weight, and Estimated Delivery Date for tracking responses	Implemented (Milestone 5)
Agent Action and Subagent (Agentforce)	Connects the Flow to the agent for conversational tracking	Implemented (Phase 3)
Record-Triggered and Scheduled Flows	Parcel status updates and notifications	Planned in project scope
Validation Rules and Approval Processes	Validation of parcel inputs and approvals	Planned in project scope
Apex Classes and Batch Apex	Automated parcel operations and periodic updates	Planned in project scope
Email Alerts and Templates	Booking confirmations and delivery updates	Planned in project scope

3.4 Phase 3: UI/UX Development & Customization

This phase enables agents to efficiently manage parcel operations and ensures secure access control using permission sets and AI-driven agent experiences built with Agentforce.

The UI/UX development phase focuses on organizing the Salesforce interface used for the SwiftShip Tracker application. The main purpose of this phase is to provide a structured interface through which users can access parcel-related information and interact with the configured Salesforce components.

The Parcel Tracker application provides access to the relevant records and their associated information. The interface is organized around the Salesforce application structure so that users can navigate between the required components without unnecessary complexity.

3.4.1 Milestone 1: Creating Permission Sets

1. In Setup, type Permission Sets in the Quick Find box and click Permission Sets under Users.
2. Click New and enter Label “Swift Ship” (the API Name auto-fills) and License “Einstein Agent”, then click Save.
3. Open the permission set, go to Apps, Object Settings.
4. For each object (Parcel__c, Delivery__c, Sender__c, Receiver__c), click Edit and enable Read, Create, and Edit (Delete is optional). Under Field Permissions, ensure at least Visible is checked for all key fields.
5. Click Save after each object.
6. The **Swift Ship** permission set is used to provide the required access to the Salesforce components involved in the project. The permission set includes access to the custom objects used for parcel tracking, including **Parcel__c, Delivery__c, Sender__c, and Receiver__c**.

Figure 44: Permission Sets List

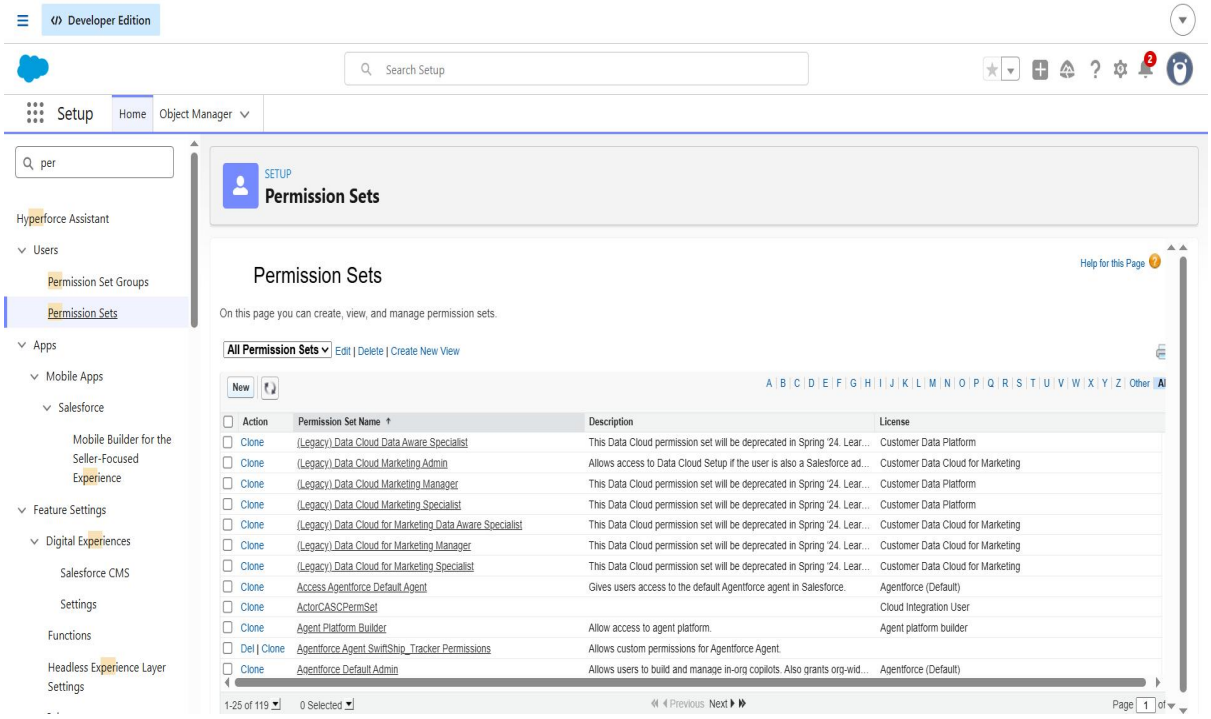


Figure 45: Swift Ship Permission Set Overview

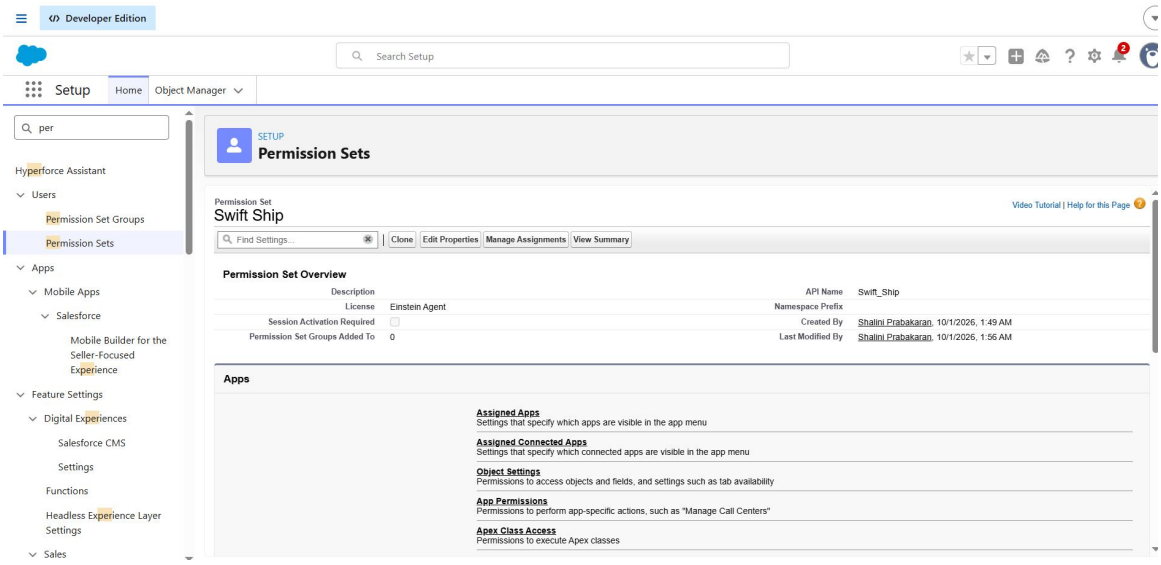


Figure 46: Permission Set: Object Settings

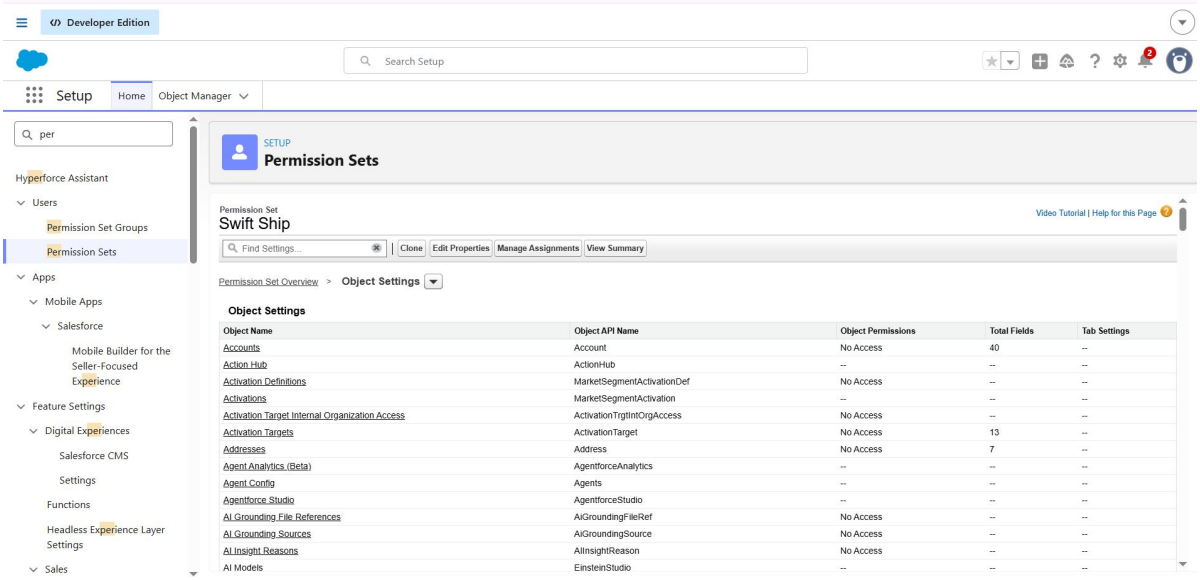


Figure 47: Parcel__c Object and Field Permissions

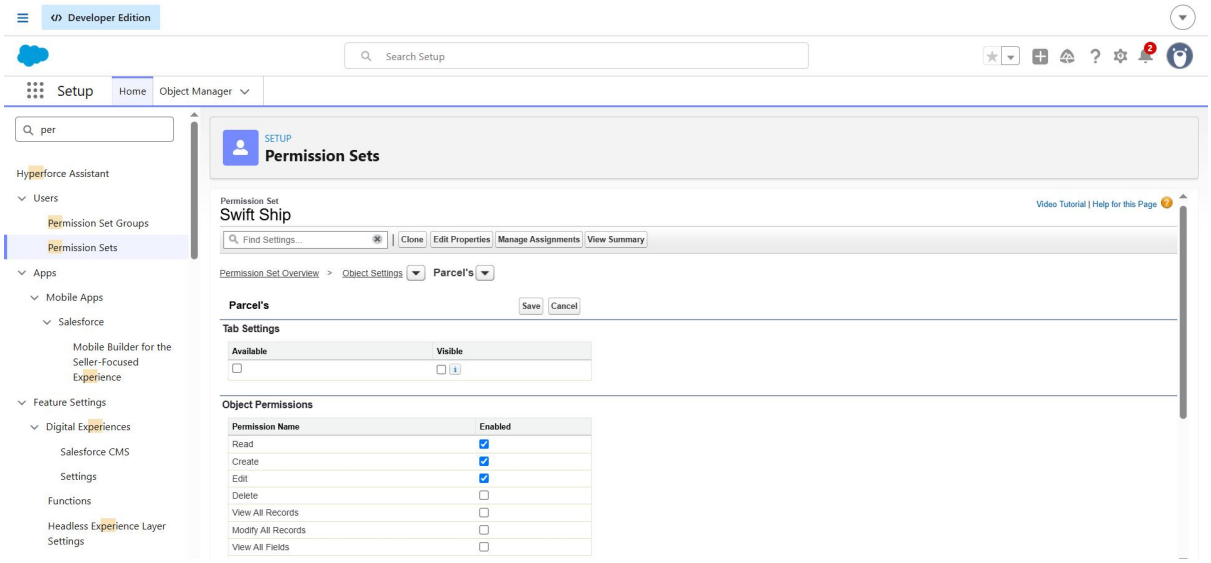


Figure 48: Delivery__c Object and Field Permissions

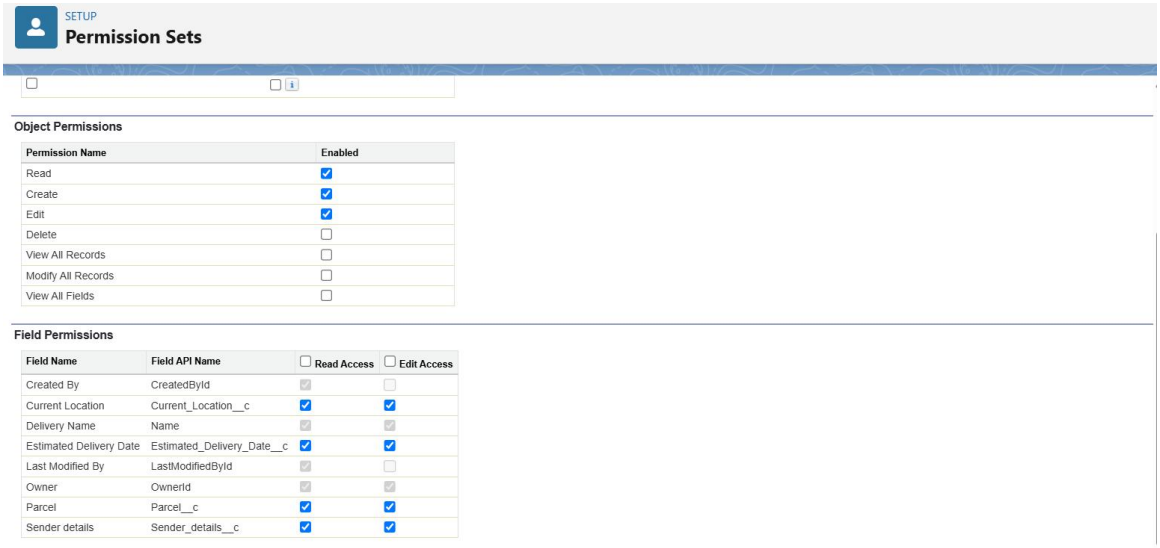


Figure 49: Sender__c Object and Field Permissions

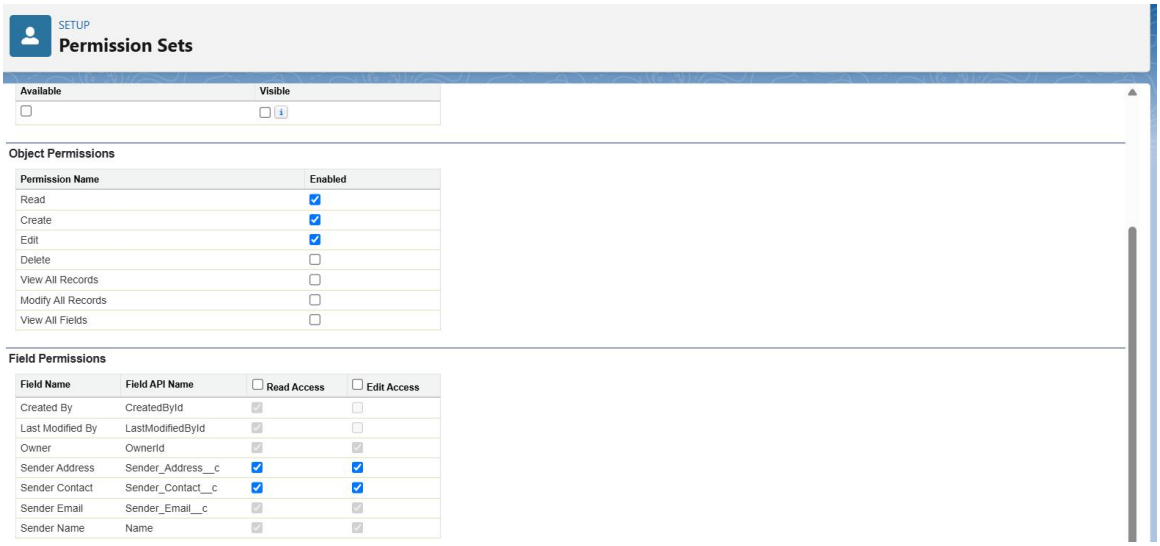
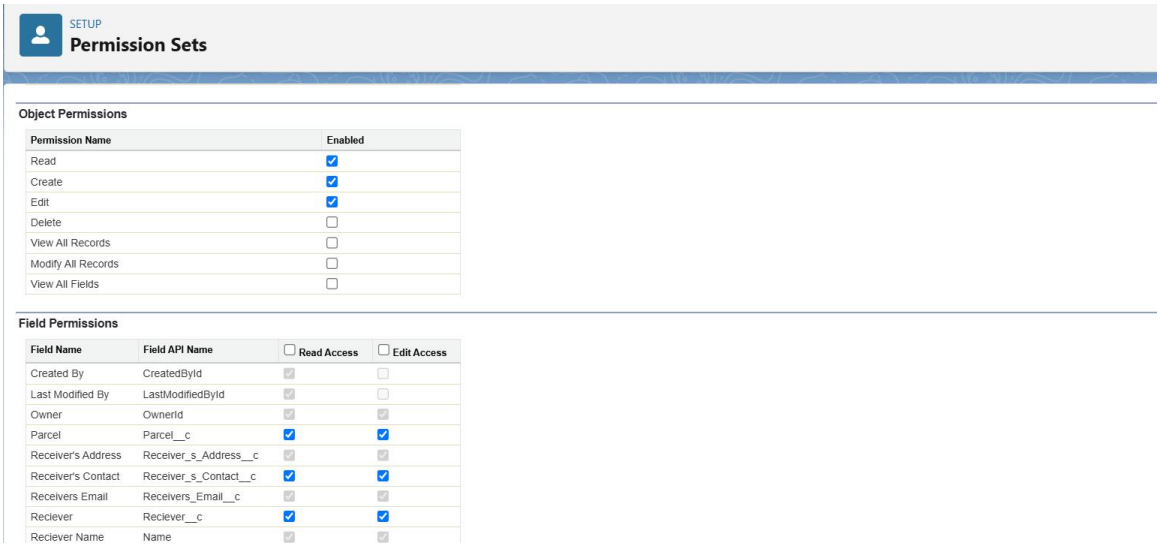


Figure 50: Receiver__c Object and Field Permissions



3.4.2 Milestone 2: Assigning Permissions to the Agent User

After configuring the permission set, it is assigned to the appropriate Einstein Service Agent user. The Agentforce Service Agent Secure Base permission set is also considered as part of the user access configuration.

This step ensures that the user responsible for executing the Agentforce functionality has the required permissions before the agent is tested. Proper permission assignment is important because the agent's ability to access Salesforce records and execute configured actions depends on the access available to its user context.

1. In Setup, type Users in the Quick Find box and click Users.
2. Open the Einstein Service Agent user that represents the agent.
3. Scroll to Permission Set Assignments and click Edit Assignments.
4. Select the “Swift Ship” permission set, click Add to move it to Enabled Permission Sets, and click Save.

Figure 51: All Users: Einstein Service Agent User

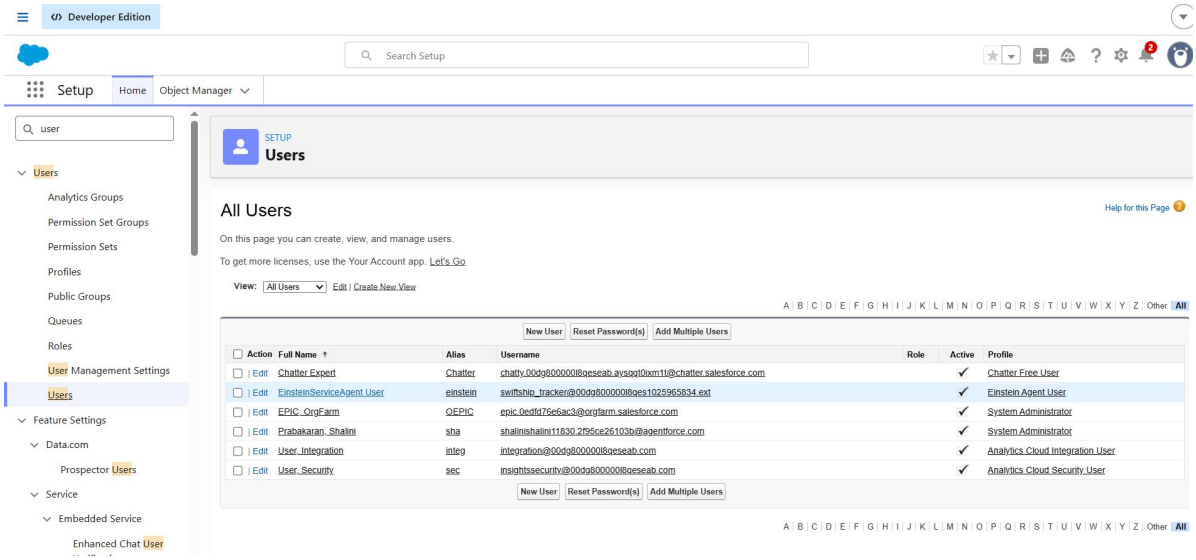


Figure 52: Edit Permission Set Assignments for the Agent User

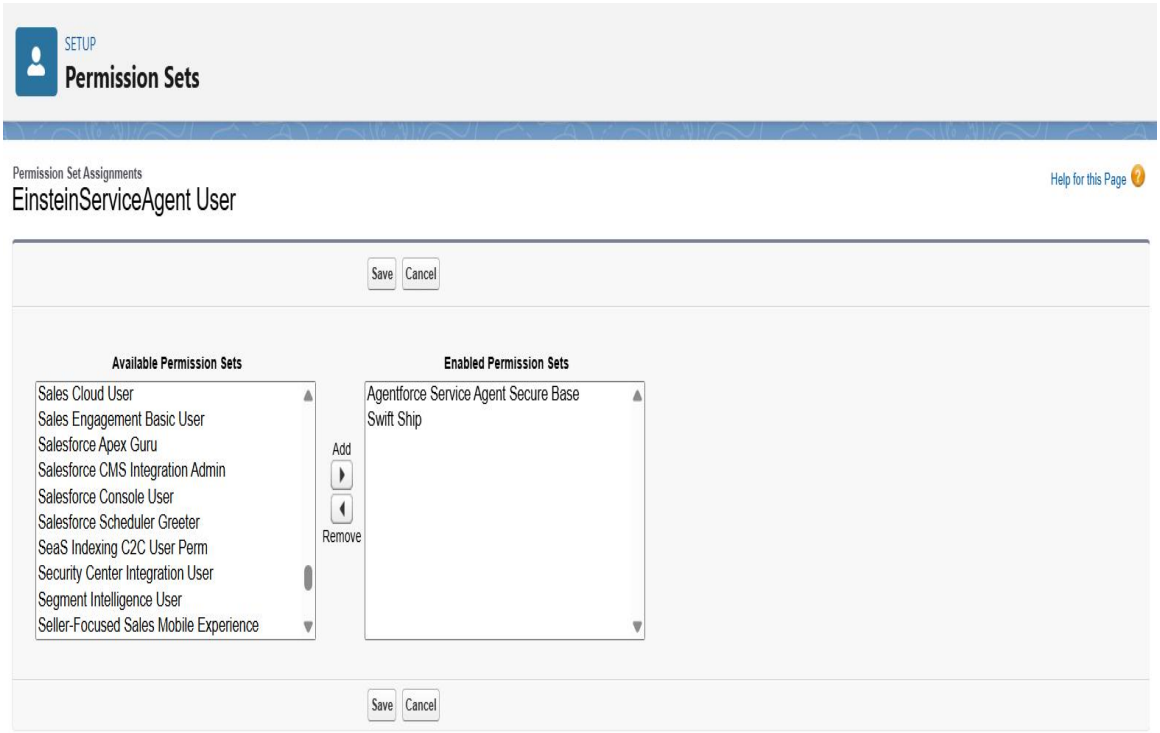
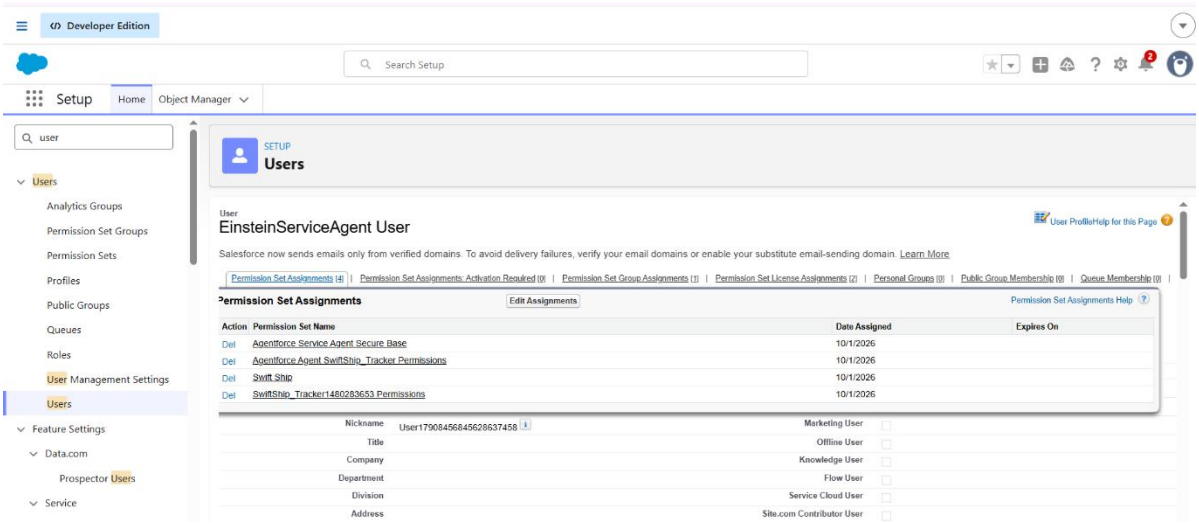


Figure 53: Permission Set Assignments of the Agent User



3.4.3 Milestone 3: Org Setup for Agentforce

The SwiftShip Tracker agent is configured to support parcel-related interactions. The agent provides the conversational interface through which users can submit their parcel-related requests.

The Parcel Details Inquiry subagent is configured as the relevant area for handling parcel information requests. This separates the responsibility of understanding and processing parcel inquiries from the overall agent configuration.

The configured structure allows the user request to move through the Agentforce components and reach the appropriate action for retrieving parcel information.

Enable the Salesforce Default Agent

1. From Quick Find, search for “Salesforce Go” and open it.
2. Search for Default Agent, click Get Started, and turn it on.
3. Refresh the page.

Enable Einstein Setup

Go to Setup, Quick Find, Einstein Setup, and click Get Started if prompted. Toggle on Turn On Einstein, Global Languages, and Deploy Prompt Templates, and then refresh the browser page. Next go to Setup, Quick Find, Agentforce Agents, and toggle on Agentforce and Enable Agentforce Default Agent.

Figure 54: Einstein Setup: Einstein, Global Languages and Deploy Prompt Templates Enabled

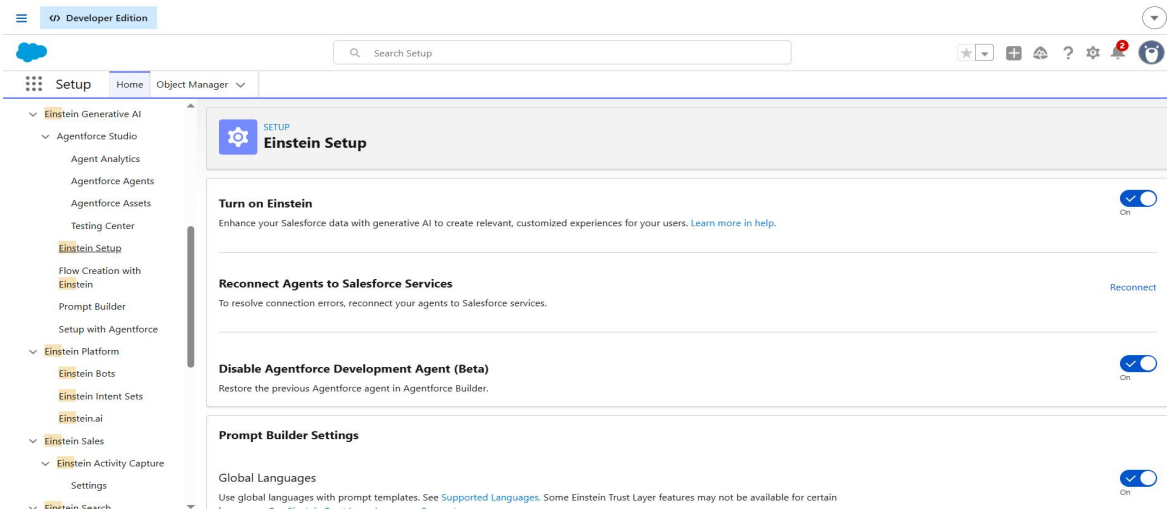
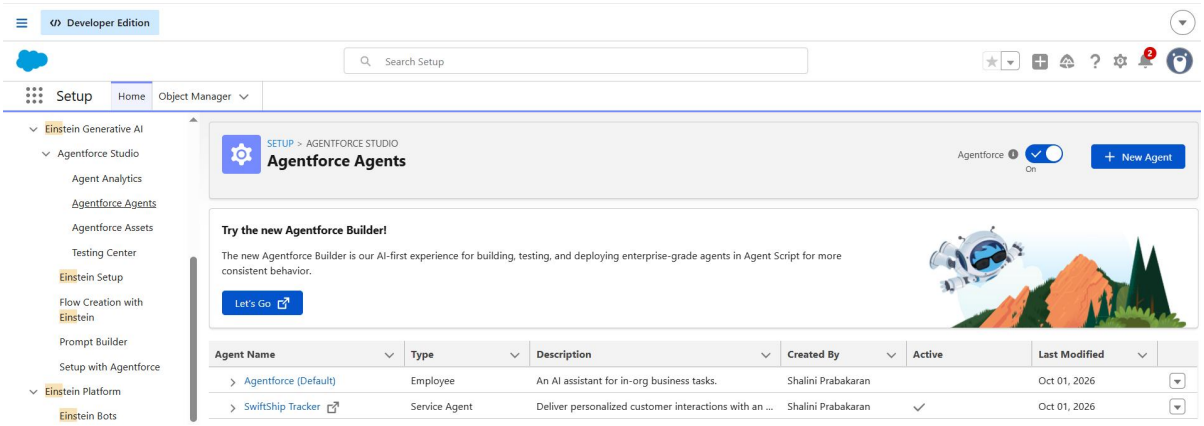


Figure 55: Agentforce Agents Setup: Agentforce and Default Agent Enabled



3.4.4 Milestone 4: Building the Agent

The fourth milestone focuses on building and configuring the SwiftShip Tracker Agent using Agentforce. This milestone connects the Salesforce data and automation components created in the earlier stages with the conversational agent. The main objective is to configure the agent so that it can handle parcel-related inquiries and use the appropriate Salesforce action to retrieve the required information.

The agent is configured with the Parcel Details Inquiry subagent, which is responsible for handling parcel-related requests. The subagent is connected to the configured Parcel Details Flow, allowing the agent to use the Salesforce Flow when parcel information needs to be retrieved.

The configuration process involves defining the agent structure, associating the required subagent, and exposing the Flow as an available agent action. This ensures that the user's

request can be processed through the appropriate sequence instead of directly accessing unrelated Salesforce components.

The overall working sequence of the agent can be represented as:

User Query → SwiftShip Tracker Agent → Parcel Details Inquiry → Parcel Details Flow → Salesforce Parcel Record → Response

During configuration, the agent is also associated with the required user permissions. The Einstein Service Agent user and the configured Swift Ship permission set provide the access required for the agent to work with the project components.

After the agent configuration is completed, it is prepared for testing in the Agentforce Preview environment. The test stage verifies whether the agent can correctly process parcel-related requests, invoke the configured action, retrieve the required information, and provide the expected response.

This milestone therefore establishes the connection between the Agentforce conversational layer, Salesforce Flow automation, Prompt Builder configuration, and parcel data. The completed agent configuration is subsequently validated through the testing activities described in Phase 4.

1. In Setup, search for Agentforce Agents in Quick Find, click Agentforce Agents, open the default agent, and click Open in Builder.
2. In Subagents, click New, then New Subagent, and create a subagent describing its job: the agent should give the parcel details related to the user by asking for the parcel ID.

Figure 56: Agentforce Agents List: Default Agent

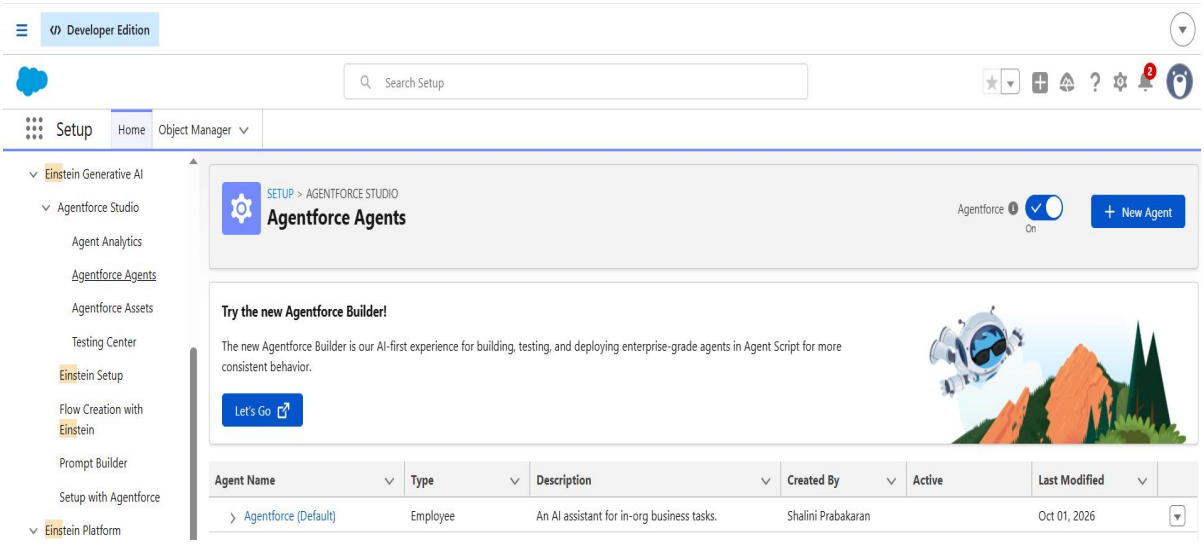


Figure 57: Agentforce (Default Agent) Details: Open in Builder

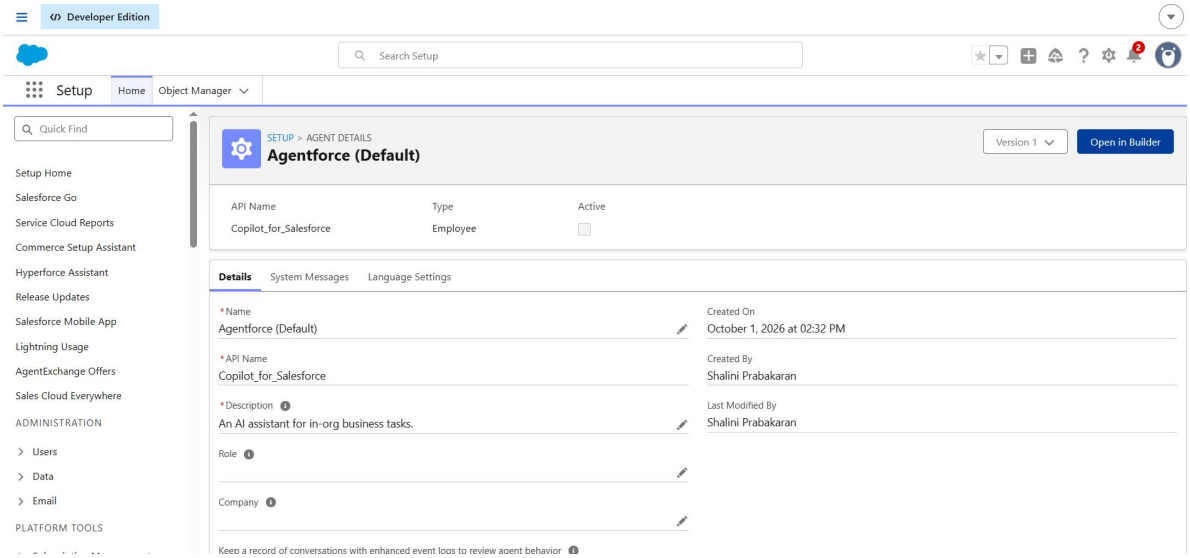


Figure 58: Agentforce Builder: Adding a New Subagent

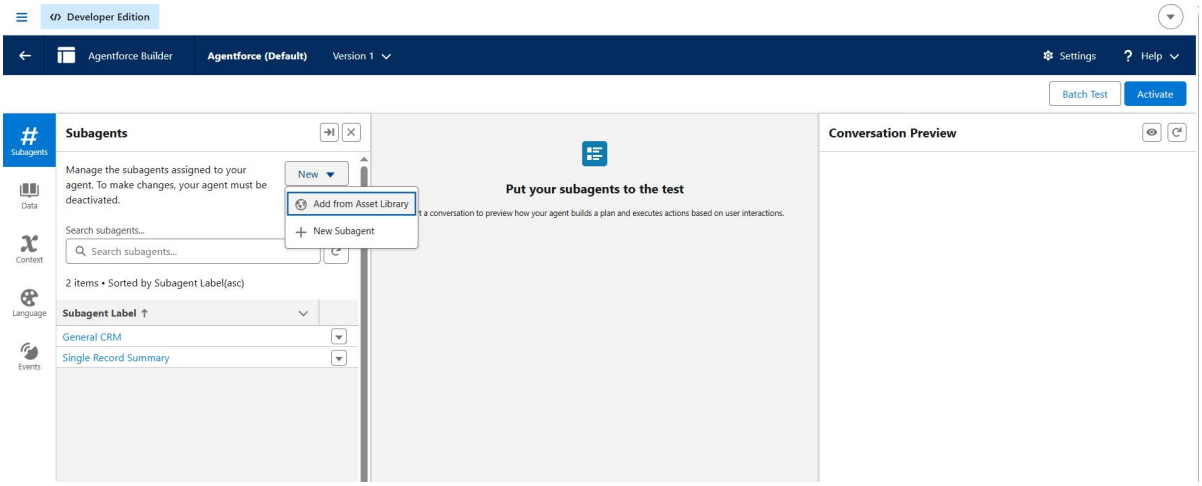
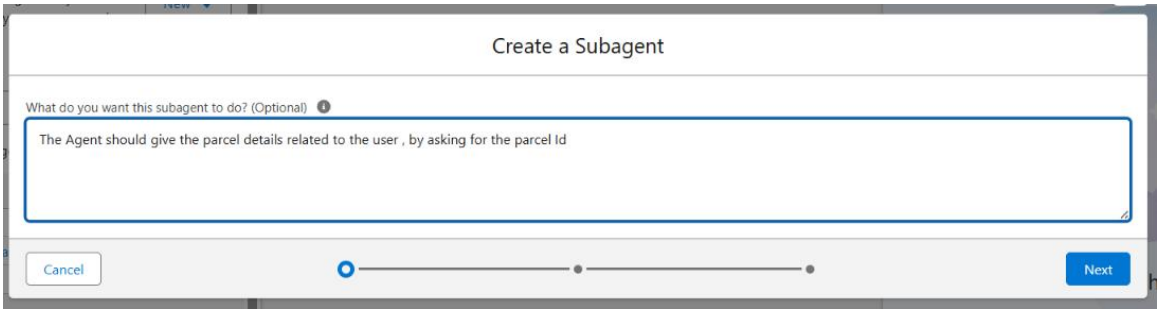


Figure 59: Create a Subagent Dialog



Adding Actions to the Agentforce Asset Library

1. In Agentforce Assets, open the Actions tab and click New Agent Action.
2. For Reference Action Type, select Flow and click Next.
3. For Reference Action, select the recently created Flow (Parcel Details), click Next, and fill in the agent action configuration.

Figure 60: Agentforce Asset Library: Actions

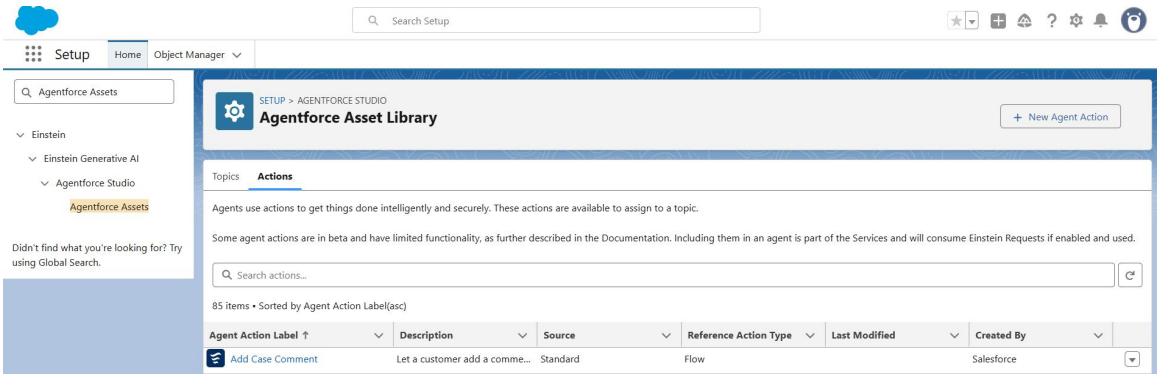


Figure 61: Create an Agent Action: Reference Action Type

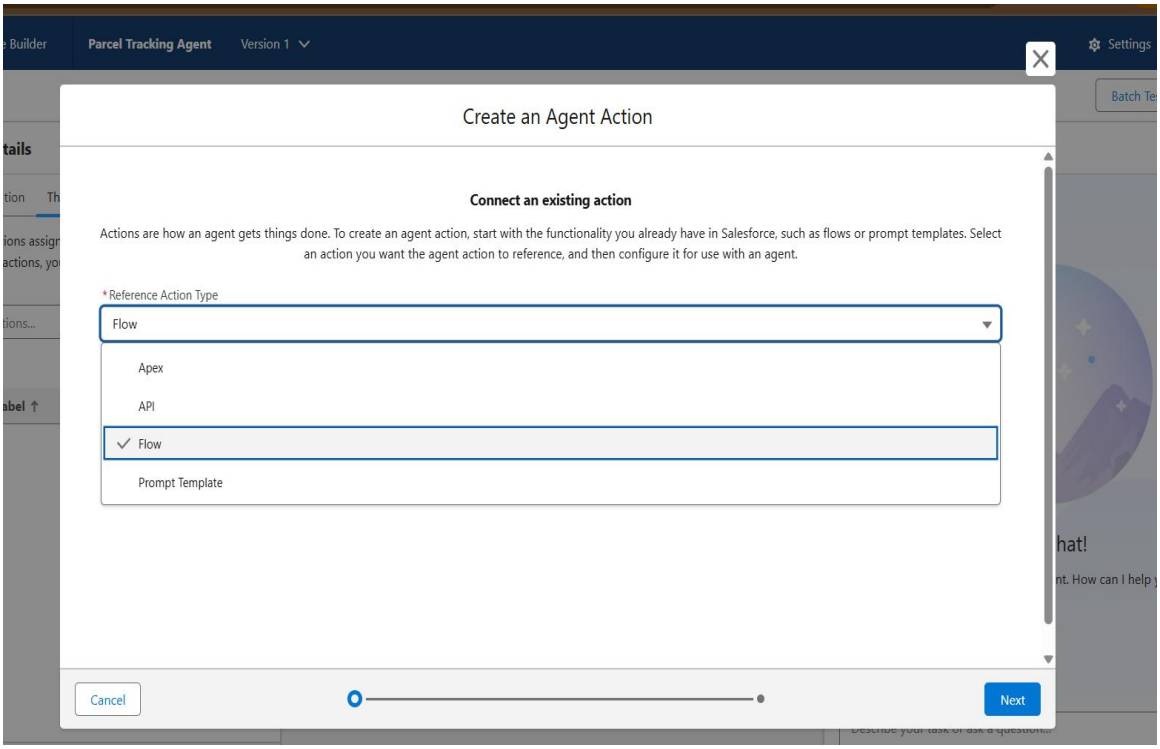
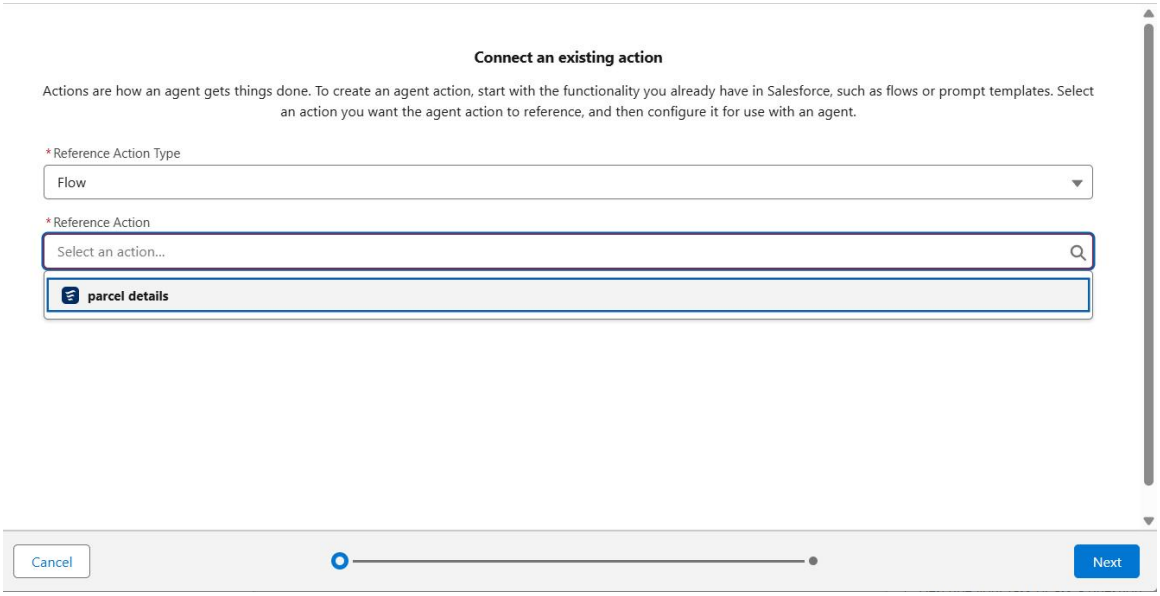


Figure 62: Create an Agent Action: Reference Action (Parcel Details Flow)



Assigning the Action to the Subagent

1. In Agentforce Builder, open the Parcel Details subagent, go to This Subagent's Actions, and click New, then Add from Asset Library.
2. Select the Flow action that was created (Parcel Details) and click Finish.
3. Click Settings and rename the agent to "SwiftShip Tracker", then click Save.
4. Click Activate; if the Configuration Issues Detected dialog appears, click Ignore & Activate.

Figure 63: Subagent Actions: Add from Asset Library

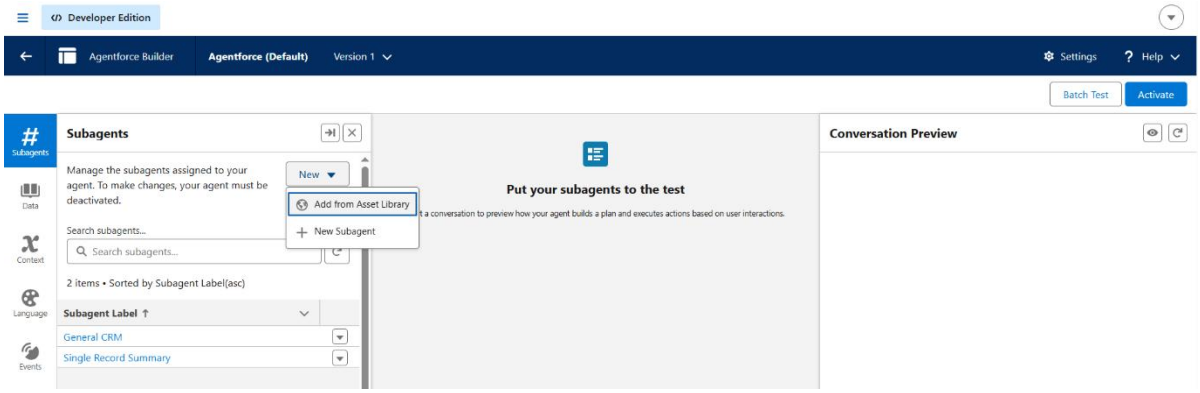


Figure 64: Add Available Actions: Parcel Details Selected

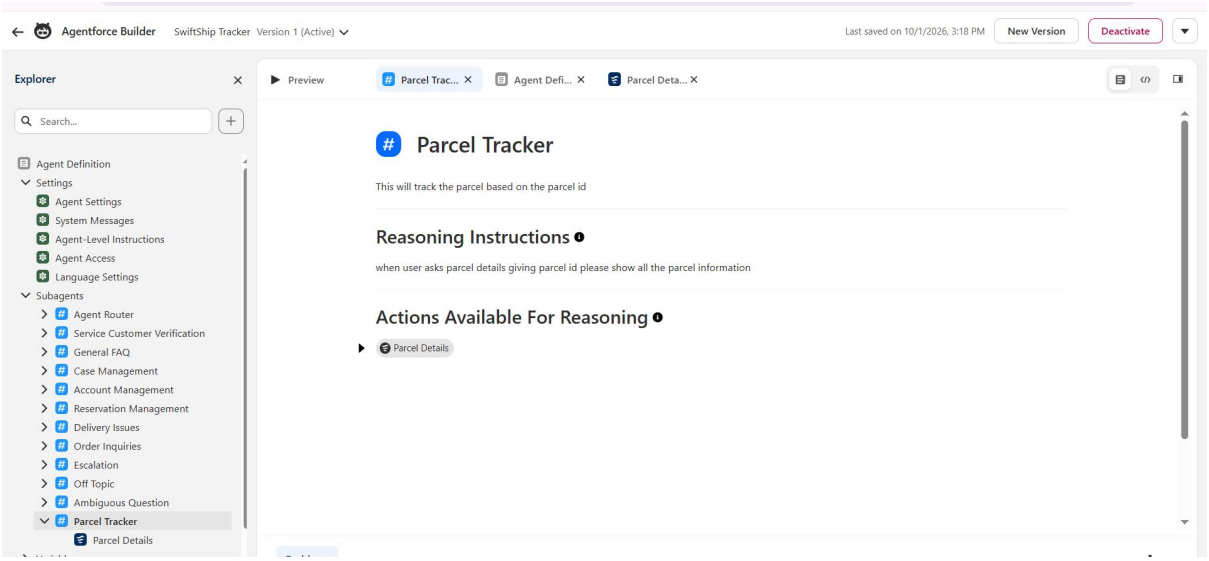


Figure 65: Subagent Actions: Parcel Details Added

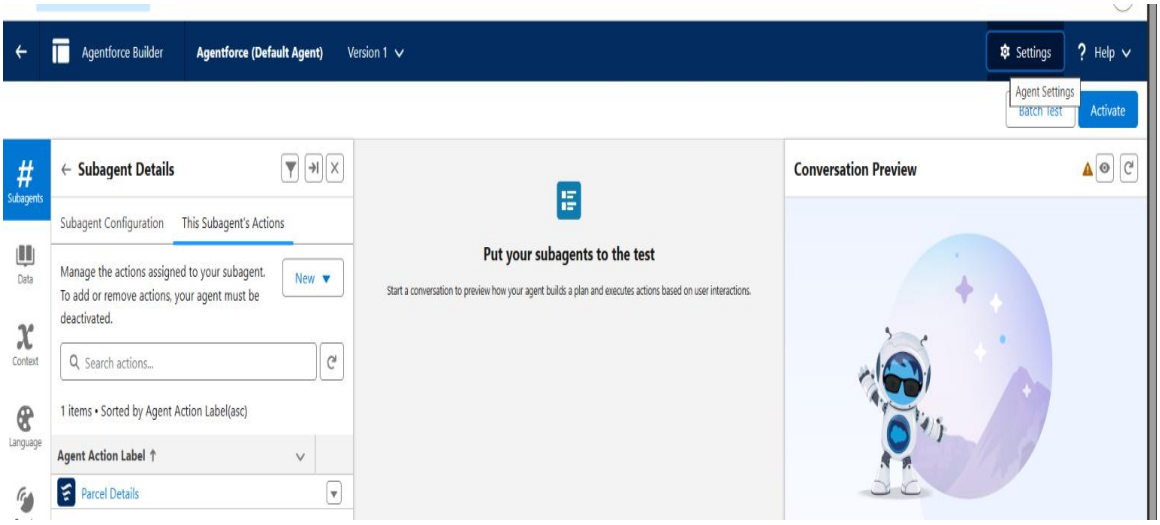


Figure 66: Edit Agent Details: Agent Renamed to SwiftShip Tracker

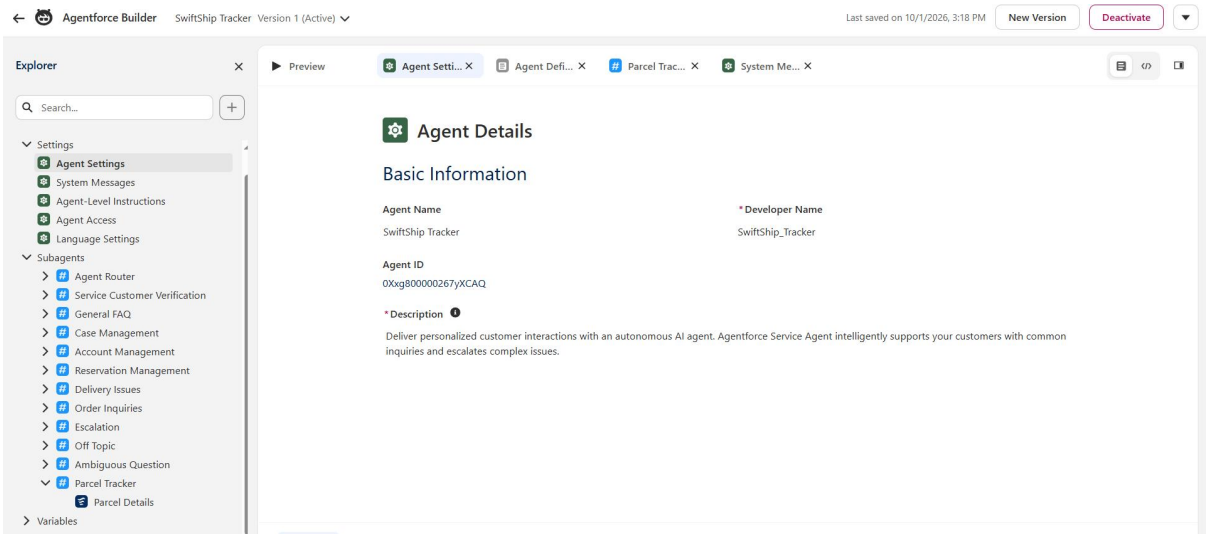
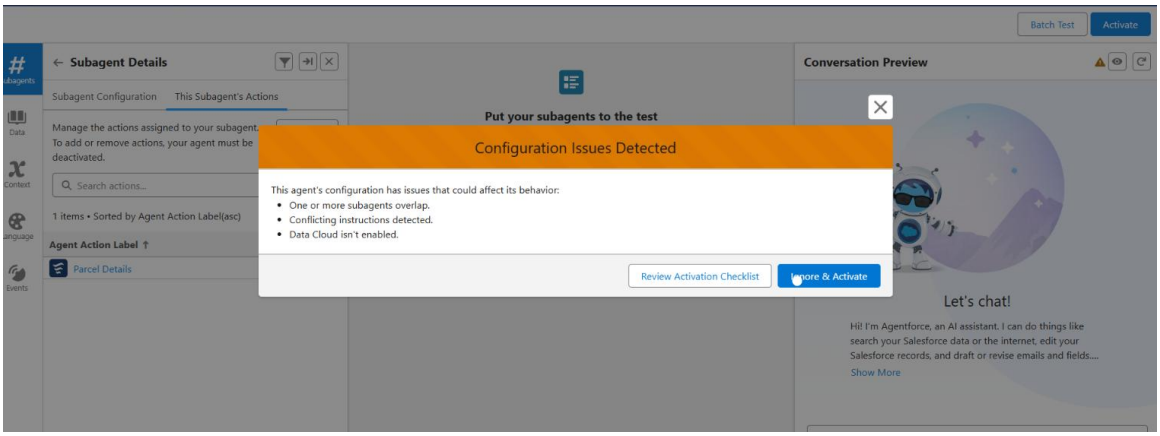


Figure 67: Configuration Issues Detected: Ignore & Activate



3.5 Phase 4: Data Migration, Testing & Security

This phase validates the functionality and reliability of the agent created in Agentforce Builder. It ensures that the agent correctly recognizes topics, triggers the assigned Flow, retrieves parcel data, and performs the intended actions without access or permission issues.

3.5.1 Test Data

Phase 4 begins with the preparation of sample parcel data required for testing the SwiftShip Tracker system. The test data is created using the same fields and object relationships configured during the implementation phase. A sample Parcel ID such as P-123 is used to verify whether the system can correctly retrieve and display parcel-related information. The test records include details such as parcel status, weight, estimated delivery date, sender information, and delivery-related information.

The purpose of preparing test data is to ensure that the Agentforce agent, Prompt Builder, and Flow work together using actual Salesforce records. Instead of testing the components individually, the complete process is verified using connected data.

SwiftShip Tracker was built in a Salesforce Developer Org. Sample Parcel, Sender, and Receiver records were created in the org and used to validate the agent, the prompt template, and the Flow. The sample parcel used for testing has Parcel ID P-123.

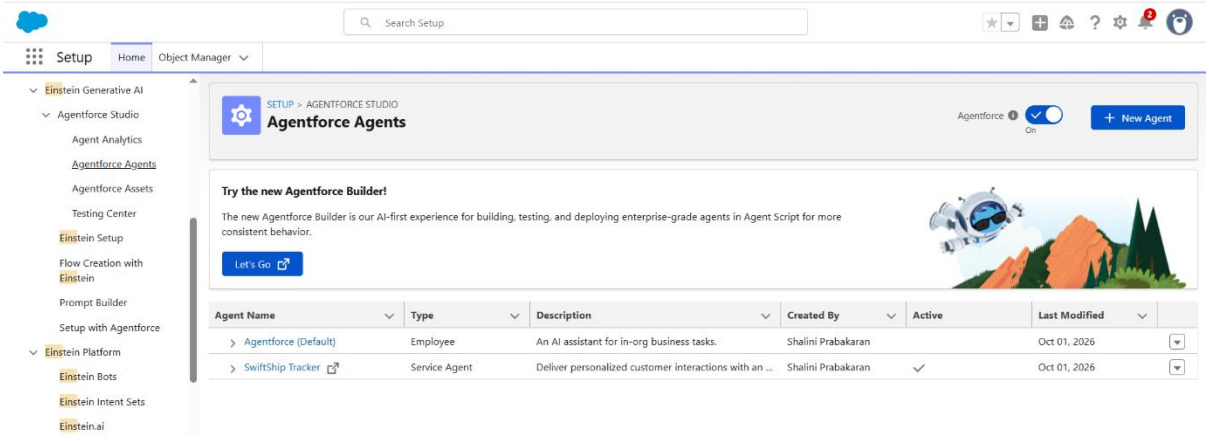
3.5.2 Verify the Agent

In Setup, open Agentforce Agents and confirm that the agent (SwiftShip Tracker) appears in the list and is active.

After creating the required test records, the SwiftShip Tracker Agent is verified through the Agentforce interface. The user provides a parcel-related query, and the agent processes the request through the configured subagent and associated action.

The verification focuses on whether the agent understands the parcel inquiry, identifies the required action, retrieves the correct record, and provides the relevant parcel details. This step helps confirm that the configuration completed in the earlier phases is functioning as an integrated system.

Figure 68: Agentforce Agents List: SwiftShip Tracker Agent Active



3.5.3 Validate the Assigned Subagent

In Agentforce Builder, the subagent “Parcel Details Inquiry” was confirmed. Its classification description states that it handles user requests to retrieve parcel details by asking for the parcel ID, and its scope limits it to obtaining the parcel ID from the user and retrieving the relevant information. The Parcel Details action (Flow) is linked to this subagent.

The **Parcel Details Inquiry** subagent is validated to ensure that it performs its intended responsibility. The subagent is associated with the required Flow action so that parcel information can be retrieved when a customer makes an inquiry.

During validation, attention is given to the connection between the user's request, the subagent, the Flow, and the Salesforce records. The response is checked against the original parcel record to make sure that the information returned by the system corresponds to the correct data.

The response generated by the system is then compared with the original parcel record stored in Salesforce. Important details such as the **Parcel ID, Status, Weight, and Estimated Delivery Date** are checked for consistency. This comparison helps confirm that the subagent is retrieving information from the correct record and presenting the expected parcel details to the user.

The validation also helps identify possible configuration issues such as incorrect Flow association, missing input values, or improper record mapping. By verifying these connections before completing the testing phase, the system can ensure that the Parcel Details Inquiry subagent works as an integrated part of the SwiftShip Tracker Agentforce workflow.

Figure 69: Subagent Configuration: Parcel Details Inquiry

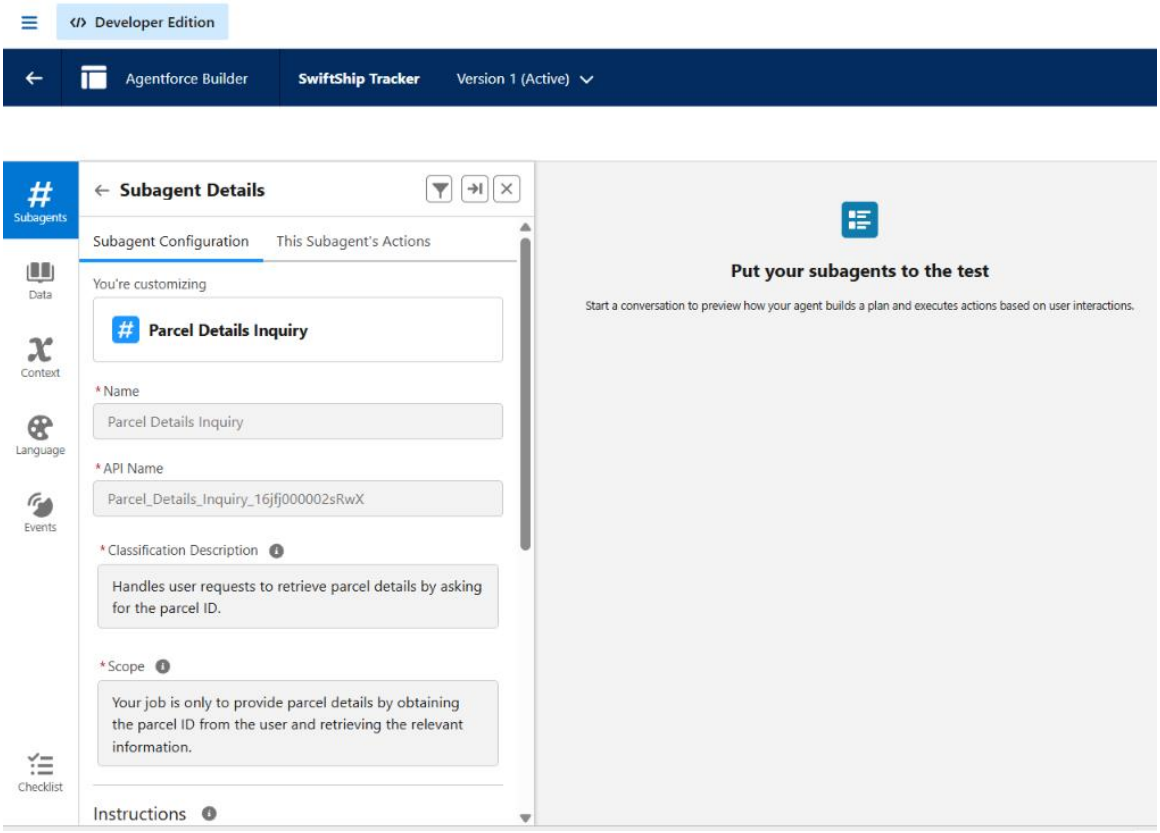
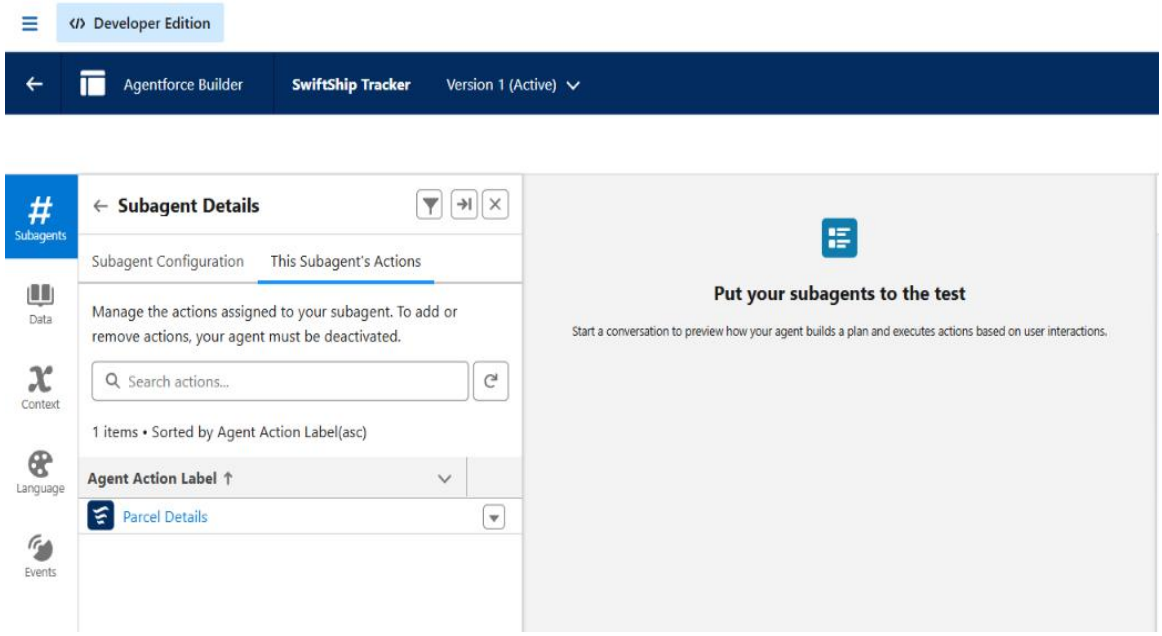


Figure 70: Subagent Actions: Parcel Details Flow Linked



3.5.4 Check the Flow Association

In Setup, Flows, the Parcel Details Flow was checked and confirmed to be active. The Flow handles Parcel ID input, parcel details retrieval, and returns the prompt response. Where data access is restricted, the Flow can be run in System Context (Without Sharing) from its Advanced Settings.

The **Parcel Details Flow** plays an important role in the testing process. The Flow receives the required input through the configured ids variable and uses Salesforce record information to retrieve parcel details.

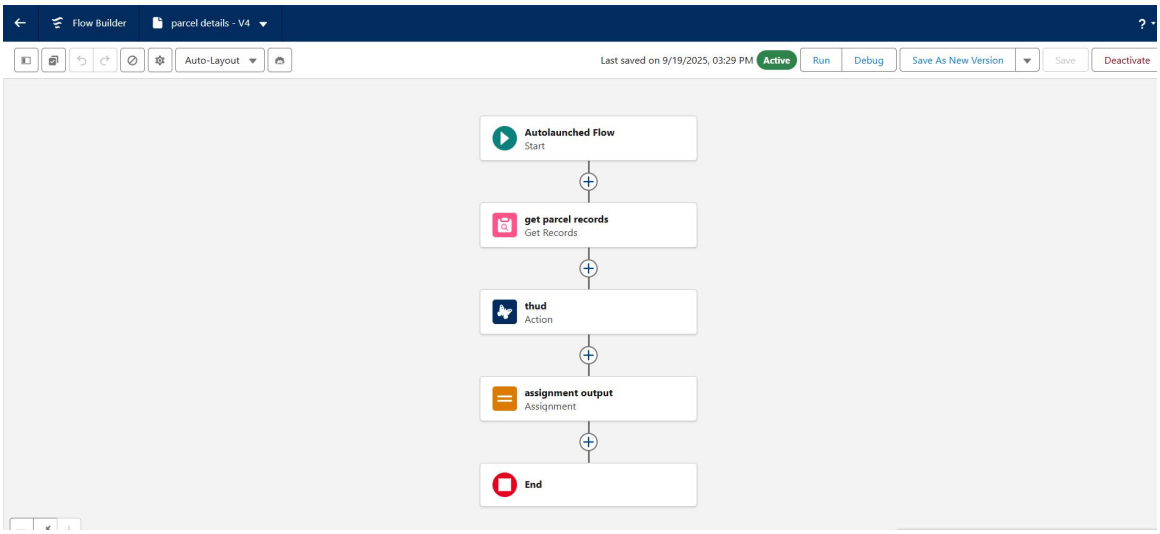
The Flow execution is checked to confirm that the **Get Records** element retrieves the appropriate parcel information. The configured Prompt Builder action is then used to process the retrieved record details. Finally, the Assignment element prepares the output that can be used by the Agentforce agent.

This validation ensures that data moves correctly through every stage of the automation process without breaking the configured sequence.

The validation also helps identify configuration-related issues such as incorrect input values, improper record retrieval, missing field information, or incorrect action mapping. By checking each stage individually and then validating the complete Flow execution, the reliability of the configured automation can be confirmed.

Overall, this validation ensures that data moves correctly from the user request through the Flow, Salesforce record retrieval, Prompt Builder processing, and final output without breaking the configured sequence. It confirms that the **Parcel Details Flow** functions correctly as an important part of the overall SwiftShip Tracker Agentforce workflow.

Figure 71: Parcel Details Flow Structure



3.5.5 Activate and Test the Agent

The agent was activated and tested in Preview mode. Example queries for testing are "Track parcel ID P12345" and "Update delivery status for parcel P67890 to Delivered." The figure below shows the test conversation recorded for this project.

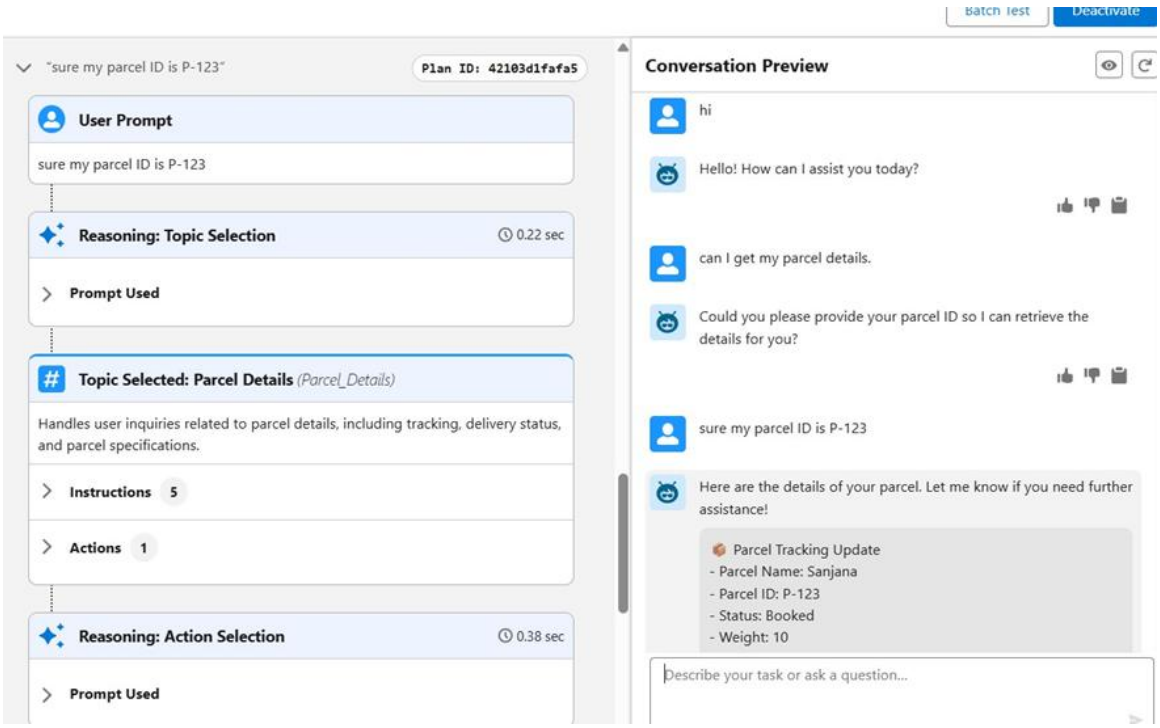
During the testing process, example queries such as **"Track parcel ID P12345"** and **"Update delivery status for parcel P67890 to Delivered"** were used to check whether the agent could understand the user's request and process it through the configured Agentforce components. The responses generated by the agent were observed to verify whether the appropriate action was triggered for the given request.

The testing also helped verify the connection between the agent, the configured subagent, Flow, and Salesforce parcel information. The conversation was reviewed to ensure that the agent followed the expected interaction flow and returned information based on the configured project implementation.

The Preview mode provided an opportunity to identify any configuration issues during agent interaction and to verify that the implemented components were working together as expected. The test results were compared with the available Salesforce parcel information to confirm the correctness of the response.

The figure below shows the **test conversation recorded for this project**, demonstrating the interaction with the SwiftShip Tracker Agent during the validation process. This test conversation serves as evidence that the configured agent was activated and tested as part of the project implementation.

Figure 72: Agent Conversation Preview and Reasoning Trace



3.5.6 Validate Data in Salesforce

After the agent runs the Flow, the Parcels records can be opened from the App Launcher to verify the stored values. The Parcel Tracker application also displays related records such as the Receiver record shown below.

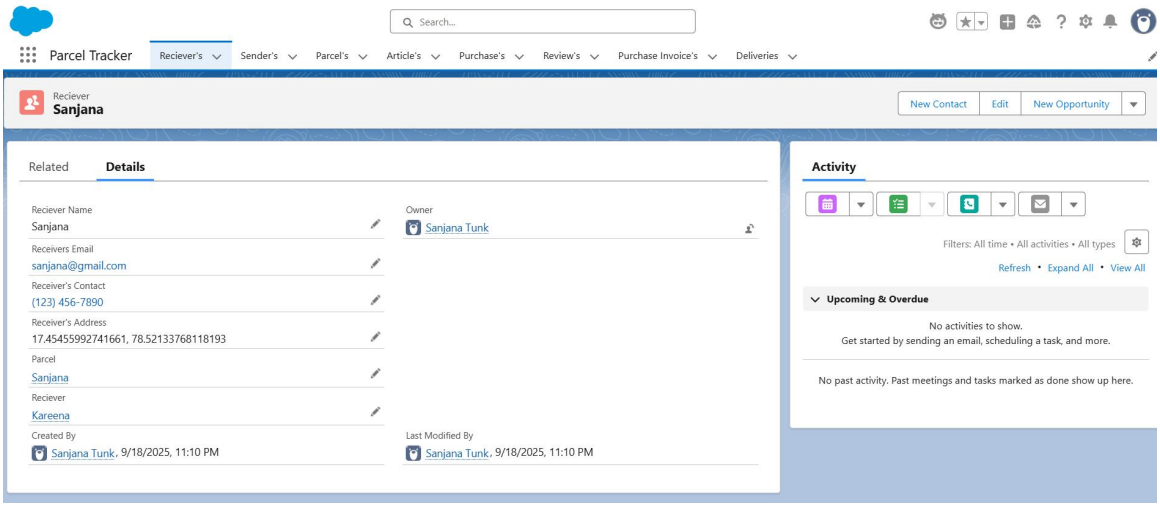
The stored Parcel record is reviewed to check the relevant information entered and retrieved during the testing process. Details such as the **Parcel ID, Status, Weight, and Estimated Delivery Date** can be compared with the information displayed during the agent interaction. This provides an additional level of validation by checking the source record directly in Salesforce.

The **Parcel Tracker** application also provides access to related records associated with the parcel. For example, the corresponding **Receiver** record can be opened and reviewed to verify that the relationship between the parcel and receiver information is available correctly.

This validation helps confirm that the Agentforce interaction, Flow execution, and Salesforce records are working together as expected. It also provides a way to verify the stored information independently of the agent response and ensures that the required project data can be accessed through the Salesforce application.

The figure below shows the Salesforce record and related Receiver information used during the validation process. This serves as evidence that the required parcel and related records are available in the implemented **Parcel Tracker** application.

Figure 73: Receiver Record in the Parcel Tracker Application



3.5.7 Functional Testing Summary

Test	Input	Observed Result
Prompt template preview	Parcel record selected in Prompt Builder preview	Generated response lists Parcel Name, Parcel ID P-123, Status Booked, Weight 10, and the Estimated Delivery Date
Agent greeting	“hi”	Agent replies: “Hello! How can I assist you today?”
Request without Parcel ID	“can I get my parcel details.”	Agent asks the user to provide the parcel ID
Request with Parcel ID	“sure my parcel ID is P-123”	Topic “Parcel Details” is selected, the Flow action runs, and the agent returns a Parcel Tracking Update with Parcel Name, Parcel ID P-123, Status Booked, and Weight 10
Record view	Receiver record opened in the Parcel Tracker app	Receiver email, contact, address, and linked Parcel and Sender lookups are displayed

3.5.8 Security Validation

- The “Swift Ship” permission set grants Read, Create, and Edit access on Parcel__c, Delivery__c, Sender__c, and Receiver__c, with field permissions set for key fields.
- The permission set is assigned to the Einstein Service Agent user, together with the Agentforce Service Agent Secure Base permission set.

- Access to the Parcel Details Flow is restricted to the enabled profiles using the Flow user access setting.
- The role hierarchy, profiles, sharing rules, and field-level security design is described in Section 3.2.6.

During validation, the configured permissions and access settings were reviewed to ensure that the Agentforce user has the required access to execute the intended parcel-tracking functionality. The validation also confirms that security settings are considered alongside the functional configuration rather than treating access control as a separate activity.

Overall, the security validation helps ensure that the implemented Salesforce objects, fields, Flow, and Agentforce components operate within the access structure defined for the SwiftShip Tracker project.

3.5.9 Error and Configuration Verification

During testing, the configuration of the connected components is also reviewed. The purpose is to identify issues such as incorrect field mapping, missing record information, incorrect Flow inputs, or improper action association.

Particular attention is given to the relationship between the Agentforce agent and the **Parcel Details Inquiry** subagent, as well as the association between the subagent and **Parcel Details Flow**. These connections are essential for successfully processing parcel-related requests.

3.5.9 End-to-End Testing Summary

The final stage of Phase 4 combines all the previously tested components into a single workflow:

User Query → Agentforce Agent → Parcel Details Inquiry Subagent → Parcel Details Flow → Salesforce Record → Prompt Builder → Agent Response

This complete flow demonstrates how the different Salesforce components work together to provide parcel tracking information. Testing the entire sequence provides stronger validation than testing each component independently.

3.5.10 Phase 4 Outcome

At the end of Phase 4, the SwiftShip Tracker implementation is validated through test data, Flow execution, Prompt Builder processing, subagent verification, and Agentforce interaction. The testing confirms whether the configured components are properly connected and whether parcel information can be retrieved and presented through the agent.

This phase therefore acts as the validation stage of the project, ensuring that the configuration completed in the earlier phases can be used as an integrated parcel tracking workflow.

4. Phase 5: Deployment, Documentation & Maintenance

4.1 Deployment Strategy

The purpose of this phase is to validate that the agent, created using Salesforce Agent Builder, works as intended within the Salesforce Developer Org environment. The main goal is to understand how an AI-powered service agent can be tested to ensure correct topic recognition, Flow execution, and data retrieval from Salesforce custom objects. Testing is conducted within the Developer Org to simulate how the agent would perform in a live setup, ensuring that permissions, flows, and interactions function correctly before any deployment stage.

Real-world production testing (UAT or sandbox deployment) is not performed in this project because:

- Developer Edition orgs are standalone environments and do not support production-linked testing environments.
- Enterprise-level Agentforce deployment involves sandbox testing, staging environments, and CI/CD automation, which are outside the scope of this academic/prototype implementation.

4.2 Maintenance, Monitoring & Troubleshooting

In the SwiftShip Tracker project, ongoing maintenance ensures that Parcel, Customer, and Delivery data remain accurate, synchronized, and up to date. Regular monitoring helps detect issues such as misrouted agent queries, failed Flow executions, or performance delays in Agentforce responses. Troubleshooting activities include resolving configuration errors, updating Flow logic, refining agent topics, and improving prompts to enhance user interactions.

Although this project is implemented within a Salesforce Developer Org for learning purposes, in real-world enterprise deployments these maintenance and monitoring practices are essential. They ensure that AI-powered service agents remain reliable, scalable, and continuously aligned with evolving customer service operations and business goals.

4.3 Troubleshooting Considerations

If the agent does not return the expected parcel information, the configured components can be checked systematically. The Parcel record can first be verified to ensure that the required data is available. The Parcel Details Flow can then be reviewed to confirm that the input and Get Records configuration are correct.

The Prompt Builder configuration can also be checked to ensure that the required Parcel resources are correctly mapped. Finally, the Agentforce subagent and action association can be reviewed to ensure that the correct Flow is being invoked.

This step-by-step approach makes it easier to identify configuration-related issues without changing unrelated components of the system.

4.4 Documentation Summary

This document records the purpose, data model, configuration, automation, and testing of SwiftShip Tracker. It covers the business overview and objectives, the phase-wise implementation (requirement, development, UI, testing, and security), the data model with ER diagram, the automation components, the security model, and the testing results supported by screenshots of the Salesforce configuration. It is intended to support future development, administration, training, and knowledge transfer.

4.5 Project Documentation

Documentation is an important part of the SwiftShip Tracker project because it provides a structured explanation of the complete implementation process. The documentation covers the project objectives, requirement analysis, system design, Salesforce configuration, automation components, Agentforce implementation, testing, security validation, and project outcomes.

The documentation also provides explanations of the major Salesforce components used in the project, including **Parcel__c, Delivery__c, Sender__c, Receiver__c, Parcel Details Flow, Prompt Builder, and Agentforce**.

Screenshots and diagrams are included wherever appropriate to provide visual evidence of the configuration and implementation. These materials help explain the actual steps followed during development and validation.

5. Conclusion

5.1 Project Learnings and Outcomes

SwiftShip Tracker demonstrates how Salesforce can be used to build a centralized parcel management and tracking solution. The project integrates custom objects, relationships, Flows, Agentforce AI, Prompt Builder, and security controls to support parcel tracking and customer communication. Apex, Email Alerts, Experience Cloud, and reporting are included in the project scope and design.

The integration of Agentforce AI with Prompt Builder and Salesforce Flow provides a conversational approach to parcel tracking, enabling customers to obtain shipment information without depending entirely on manual support. The project therefore provides a practical foundation for a scalable and user-friendly parcel management solution.

5.2 Advantages

- Centralized parcel and customer information in Salesforce.
- Better visibility into parcel booking and delivery tracking.
- Automates key logistics processes using Salesforce Flow.
- Agentforce enables conversational customer support and parcel-status assistance.
- Prompt Builder supports AI-powered responses and self-service.
- Reduces manual effort for customers, support teams, and administrators.
- Provides timely notifications and updates during the delivery process.
- Experience Cloud can provide customers with convenient access to tracking information.
- Supports integration with external logistics systems.
- Can scale to support multiple locations and growing parcel operations.

5.3 Limitations

- Requires accurate and up-to-date parcel and customer data.
- Dependence on Salesforce configuration and automation.
- Agentforce responses depend on properly configured prompts and available data.
- External logistics integrations may increase implementation complexity.
- Flow and automation logic require regular maintenance when business processes change.
- Users need appropriate Salesforce or Experience Cloud access to use the system.
- Initial Salesforce implementation and configuration can require significant effort.
- Expanding the system to additional locations or integrations may require further customization.

5.4 Future Scope

- Extend Agentforce capabilities for more parcel-management queries.
- Enhance real-time delivery tracking.
- Expand customer self-service through Experience Cloud.
- Add more advanced delivery-performance analytics.
- Integrate additional external logistics systems.
- Improve AI prompts and conversational experiences.
- Introduce enterprise-level sandbox and CI/CD deployment.
- Expand automation for additional parcel operations.

5.7 Key Takeaways

- Salesforce provides a powerful platform for building scalable, intelligent, and secure business solutions.
- Agentforce AI and Prompt Builder enable conversational interactions that enhance customer experience.
- A well-designed data model and automation are critical for system reliability and performance.
- Security must be considered at every layer of the solution.
- Testing and validation are essential to ensure that integrated components work together.
- Documentation is key to knowledge transfer and future development.

5.8 Recommendations

Based on the project experience, the following recommendations are made for future work:

1. **Expand Testing:** Conduct testing in a sandbox environment with realistic data volumes.
2. **Implement Planned Features:** Complete the implementation of Apex classes, Email Alerts, and Experience Cloud.
3. **Enhance AI:** Continuously refine prompts and expand subagent topics.
4. **Monitor Performance:** Use Salesforce monitoring tools to track system performance and address issues.
5. **Gather Feedback:** Collect user feedback to identify areas for improvement.
6. **Plan for Scale:** Design for scalability from the outset to accommodate future growth.

Appendix

Ref	Category	Items
A	Custom Objects	Parcel, Delivery, Sender, Receiver
B	Standard Objects	Account, Contact, Case, Task, EmailMessage, User
C	Automation	Flows, Approval Processes, Email Alerts
D	Apex	Apex Classes, Batch Apex
E	AI	Agentforce AI, Prompt Builder
F	UI	Lightning App, Lightning Record Pages, Experience Cloud
G	Security	Profiles, Roles, Permission Sets, Sharing Rules, Field-Level Security
H	Analytics	Reports & Dashboards
I	Testing	Flow, Agentforce, data retrieval, security and permission validation